Premier University, Chattogram

Department of Computer Science and Engineering



Thesis Report

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science in Computer Science and Engineering

Explainable Cyberbullying Detection Using Deep Learning and Transformer Models with LIME

Submitted By

Mohammad Asmual Hoque Yousha
ID: 0222210005101121

Md Mostafa Kamal Anna
ID: 0222210005101129

Supervised By

Ms. Tanni Dhoom
Assistant Professor
Department of Computer Science and Engineering
Premier University, Chattogram

October 2026

Author's Declaration of Originality

We hereby declare that the thesis entitled **"Explainable Cyberbullying Detection Using Deep Learning and Transformer Models with LIME"** is our original work and has been completed under the supervision of **Ms. Tanni Dhoom**, Assistant Professor, Department of Computer Science and Engineering, Premier University, Chattogram. We further declare that this work has not been submitted, either in full or in part, to any other institution for the award of any degree or certificate.

Mohammad Asmual Hoque Yousha
ID: 0222210005101121

Md Mostafa Kamal Anna
ID: 0222210005101129

CERTIFICATION

The thesis entitled **"Explainable Cyberbullying Detection Using Deep Learning and Transformer Models with LIME"** submitted by Mohammad Asmual Hoque Yousha, ID: 0222210005101121, and Md Mostafa Kamal Anna, ID: 0222210005101129 has been accepted as satisfactory in partial fulfillment of the degree of Bachelor of Science in Computer Science and Engineering (CSE) to be awarded by Premier University, Chattogram.

Professor S. M. Nasrul Quadir
Chairman
Department of Computer Science and Engineering
Premier University, Chattogram

Ms. Tanni Dhoom
Assistant Professor
Department of Computer Science and Engineering
Premier University, Chattogram

This work is dedicated to our families and to our supervisor, whose guidance and encouragement made this research possible.

Acknowledgement

First and foremost, we are deeply grateful to our supervisor, **Ms. Tanni Dhoom**, Assistant Professor, Department of Computer Science and Engineering, Premier University, Chattogram, for her invaluable guidance throughout this work. From the moment she recognised the potential in this project and elevated it to full thesis-level research, her insight, patience, and steady encouragement shaped every stage of the study. She pushed us to improve our accuracy, to compare our work fairly against existing research, and to be honest about its limitations, and this thesis is far stronger for her advice. We sincerely thank her for the time and support she gave us, even amid her own commitments.

We also wish to thank the Department of Computer Science and Engineering, Premier University, Chattogram, and its faculty members for providing the academic environment and the resources that made this research possible. Their teaching over the course of our degree laid the foundation on which this work was built.

Finally, we are thankful to our families and friends for their constant support and understanding during the many long hours this thesis required. Their belief in us kept us going, and we dedicate this achievement, in part, to them.

Mohammad Asmual Hoque Yousha
Md Mostafa Kamal Anna

Table of Contents

Acknowledgement **iv**

List of Figures **viii**

List of Tables **ix**

1 Introduction **1**

1.1 Research Context 2
1.2 Problem Statement 2
1.3 Research Motivation 2
1.4 Research Objectives 3
1.5 Key Contributions 3
1.6 Scope of the Study 4
1.7 Significance of the Study 4
1.8 Thesis Organisation 5

2 Literature Review **6**

2.1 Background 6
2.2 Traditional Machine Learning Approaches 7
2.3 Deep Learning Approaches 7
2.4 Transformer-Based Language Models 7
2.5 Explainability in Text Classification 8
2.6 Review of Related Studies 8
2.7 Comprehensive Summary of Related Work 10
2.8 Research Gap 11
2.9 Chapter Summary 12

3 Methodology **13**

3.1 Overview of the Approach 13
3.2 Dataset 14
3.2.1 Self-Collected Data 14
3.2.2 Supplementary Data 15
3.2.3 Label Preparation and the Binary Reduction 15
3.2.4 Dataset Statistics 15
3.2.5 Labelled Examples 16
3.3 Preprocessing 17

3.4 Training, Validation, and Test Splits . . . 18
3.5 Feature Representation and Models . . . 18
3.5.1 Logistic Regression . . . 19
3.5.2 LSTM . . . 19
3.5.3 GRU . . . 19
3.5.4 RoBERTa . . . 19
3.5.5 Rationale for the Chosen Models . . . 20
3.6 Explainability with LIME . . . 20
3.6.1 Why We Used LIME . . . 20
3.6.2 How LIME Works . . . 21
3.6.3 LIME Configuration and Output . . . 21
3.7 Evaluation Metrics . . . 22
3.8 Experimental Setup and System Specification . . . 22
3.8.1 Hardware Specification . . . 23
3.8.2 Software Specification . . . 23
3.9 Chapter Summary . . . 24

4 Results and Discussion **25**
4.1 Dataset Overview . . . 25
4.2 Model Performance on the Combined Dataset . . . 27
4.3 Confusion Matrix Comparison . . . 28
4.4 Comparison with the Base Paper (Esan et al., 2024) . . . 29
4.5 Comparison with Ogunleye and Dharmaraj (2023) . . . 30
4.6 LIME Explainability Analysis . . . 31
4.6.1 A Clear, High-Confidence Case . . . 32
4.6.2 A Strong Bullying Case Driven by One Word . . . 33
4.6.3 A Correct but Low-Confidence Case . . . 34
4.6.4 Distinguishing Disagreement from Bullying . . . 35
4.7 Error Analysis . . . 36
4.7.1 The Sarcasm Failure . . . 36
4.7.2 Other Error Patterns . . . 38
4.8 Key Findings Summary . . . 38
4.9 Discussion . . . 39

5 Social Impact and Responsible Implementation **40**
5.1 Looking Out for the People Most at Risk . . . 40
5.2 Decisions People Can Actually Question . . . 41
5.3 A Helper for Moderators, Not a Replacement . . . 41

6 Future Work **42**

6.1 Closing the Training-Pipeline Gap 42
6.2 Making the Model Sarcasm-Aware 43
6.3 Restoring Fine-Grained Bullying Types 43
6.4 Raising Accuracy with Richer Representations and Ensembles 43
6.5 Scaling Up the Dataset 44
6.6 Extending to Multiple Languages 44

7 Conclusion **45**

Bibliography **46**

Source Code **49**

List of Figures

3.1 End-to-end pipeline: data collection, cleaning and deduplication, binary labelling, stratified splitting, training of the four models, evaluation, and LIME explanation on the best model. 14
3.2 How LIME produces a word-level explanation for a single comment. . . . 21

4.1 Class distribution of the combined dataset. Class 0 is Not-Bullying (2,059 comments) and class 1 is Bullying (1,941 comments). 26
4.2 Distribution of comment length, measured in number of words. Most comments are short, with a long tail of longer ones. 27
4.3 Confusion matrices of all four models on the test set ($n = 600$), drawn on a shared colour scale for direct comparison. TN = true negative, FP = false positive, FN = false negative, TP = true positive. 28
4.4 Colab output for "Everyone hates you, just go away" (Bullying, 93.30% confidence). 32
4.5 LIME bar chart for "Everyone hates you, just go away". 33
4.6 Colab output for "Nobody asked for your stupid opinion" (Bullying, 98.42% confidence). 33
4.7 LIME bar chart for "Nobody asked for your stupid opinion", dominated by stupid. 34
4.8 Colab output for "rubbish boy" (Bullying, 67.97% confidence). 34
4.9 LIME bar chart for "rubbish boy". 35
4.10 Colab output for "I don't agree with your opinion" (Not-Bullying, 97.07% confidence). 35
4.11 LIME bar chart for "I don't agree with your opinion". Note the split of "don't" into don and t. 36
4.12 Colab output for the misclassified sarcastic comment "Wow, you're really smart... said no one ever." (Not-Bullying, 95.32%, incorrect). 37
4.13 LIME bar chart for the misclassified sarcastic comment. The positive-sounding words Wow, really, and said (red) drive the wrong Not-Bullying prediction. 37

List of Tables

1.1 Overview of the thesis chapters 4

2.1 Comprehensive summary of related work in cyberbullying and abusive-language detection 11

3.1 Dataset summary after cleaning and deduplication 16
3.2 Recorded bullying sub-types among the bullying comments (not used in the binary models) 16
3.3 Representative labelled examples from the dataset, covering all six bullying sub-types 17
3.4 Training, validation, and test split (stratified, seed $= 42$) 18
3.5 LIME "important words" output for the comment "Everyone hates you, just go away" 22
3.6 Hardware specification of the experimental environment 23
3.7 Software specification of the experimental environment 23

4.1 Performance of the four models on the combined dataset (test set, $n = 600$) 27
4.2 Per-class classification report for RoBERTa on the test set 29
4.3 Same model (RoBERTa), different datasets 30
4.4 Same dataset (Esan et al., 46,017 comments), different models 30
4.5 Four-way comparison with Ogunleye and Dharmaraj (RoBERTa) 30
4.6 Full model breakdown — our data with the Ogunleye and Dharmaraj code 31
4.7 Full model breakdown — their Dataset 3 with our code 31
4.8 Summary of RoBERTa results across all experimental settings 38

Abstract

Cyberbullying on social media platforms is a growing problem, and the volume of user-generated content makes manual moderation impractical. Automatic detection has therefore become an active research area, but most existing detectors report only aggregate accuracy and give no insight into why a comment is flagged, which limits how far they can be trusted in practice. This thesis addresses both accuracy and transparency. We assemble a binary cyberbullying dataset of roughly 4,000 English comments, combining about 1,000 comments collected from YouTube, Reddit, and Facebook with about 3,000 derived from the Jigsaw toxic-comment corpus, and reduce every comment to a Bullying or Not-Bullying label. Four models are trained and compared under identical conditions: a TF–IDF Logistic Regression baseline, an LSTM, a GRU, and a fine-tuned RoBERTa transformer. RoBERTa performs best, reaching 92.33% accuracy and F1, compared with 87.33% for LSTM, 85.33% for GRU, and 83.33% for Logistic Regression. To validate the approach beyond a single split, the pipeline was also run on the base paper's own 46,017-comment Twitter dataset, where RoBERTa reached 89.01%, well above the base paper's reported 77.00%, and on a second study's data through a four-way comparison in which RoBERTa was the best model in every setting. The central novelty of the work is the addition of Local Interpretable Model-agnostic Explanations (LIME) to the detector, which explains each prediction at the level of contributing words; neither of the two comparison studies includes any such explainability. The LIME analysis confirms that correct predictions rest on sensible words and also exposes a genuine weakness of the model on sarcastic comments. The study contributes an accurate, cross-dataset validated, and explainable cyberbullying detector, together with an honest comparison that isolates training and implementation details as a factor affecting performance.

Keywords: cyberbullying detection, RoBERTa, LIME, explainable AI, text classification, deep learning.

CHAPTER 1

INTRODUCTION

Online communication has grown into one of the primary ways people talk to each other, and the same platforms that make this possible have also made it easy to hurt others at scale. Comment sections on YouTube, threads on Reddit, and posts shared on Facebook are read by millions every day, and a share of that content is hostile, threatening, or demeaning. Cyberbullying is the term commonly used for this kind of repeated or aggressive online behaviour directed at a person or group [12]. Unlike face-to-face confrontation, it follows the target home, can be seen by a wide audience, and often stays visible long after it was posted. For younger users in particular, the effect on mental health is well documented, and manual moderation alone cannot keep pace with the sheer volume of text produced each day [12].

This has made automatic cyberbullying detection an active area of research. The task, at its core, is a text classification problem: given a comment, decide whether it is bullying or not. Machine learning and, more recently, deep learning and transformer-based language models have pushed accuracy upward on several benchmarks. Yet a practical concern remains largely unaddressed in much of this work. Most published detectors report only aggregate scores such as accuracy or F1 and give no insight into why a particular comment was flagged. For a moderation system that may lead to a post being removed or a user being warned, this lack of transparency is a real limitation. A moderator, a platform, or an affected user has good reason to ask which words in a message drove the decision.

1.1 Research Context

The study reported in this thesis sits at the meeting point of two lines of work. The first is supervised text classification for abusive-language and cyberbullying detection, where the field has moved from hand-crafted features fed to classical classifiers, through recurrent neural networks such as LSTM and GRU, and on to pretrained transformers such as BERT and RoBERTa. The second is explainable artificial intelligence, which asks that a model's predictions be made interpretable to a human rather than treated as the output of a black box. These two threads are usually pursued separately. Detection papers chase higher accuracy; explainability papers tend to work on generic benchmarks. Bringing them together for cyberbullying detection is the space this work occupies.

We frame the problem as binary classification, separating bullying comments from non-bullying ones, and we treat interpretability as a first-class objective rather than an afterthought. Concretely, we pair a fine-tuned RoBERTa classifier [15] with Local Interpretable Model-agnostic Explanations (LIME) [18] so that every prediction can be accompanied by the words that most influenced it. This combination is what distinguishes the present work from the studies it builds on.

1.2 Problem Statement

The specific problem addressed here can be stated plainly. Existing cyberbullying detectors, including the base paper this thesis extends, achieve moderate to good accuracy but offer no explanation for their decisions and are frequently evaluated on a single dataset, which leaves open the question of how well they generalise. The base paper of this study, Esan et al. [1], reports an LSTM model reaching 77.00% accuracy on a Twitter cyberbullying dataset, with no interpretability component of any kind. The problem, then, is twofold: to build a detector that improves on this level of accuracy, and to make its predictions transparent so that the reasoning behind a decision can be inspected word by word.

1.3 Research Motivation

Two observations motivated this work. First, a large body of cyberbullying research reports strong numbers but stops there. When we surveyed related studies, we found that very few attach any form of explanation to their predictions, and the base paper we chose to extend is a clear example of this gap. A model that says "this comment is bullying"

without indicating why is difficult to trust, hard to debug, and awkward to defend if a moderation decision is challenged. Adding explainability is therefore not a cosmetic addition; it changes what the system can be used for.

Second, we were interested in whether modern transformer models genuinely outperform the recurrent architectures that dominate the older literature, and whether any such advantage holds up when the model is moved onto data it was not tuned for. This pushed us towards collecting our own multi-platform data and towards cross-dataset evaluation, so that the comparison would rest on more than a single favourable split.

1.4 Research Objectives

The objectives of the thesis are as follows.

- To assemble a cyberbullying dataset from multiple social platforms (YouTube, Reddit, and Facebook) together with a supplementary set derived from a public toxic-comment corpus, and to prepare it for binary classification.
- To implement and fairly compare four models spanning the classical-to-transformer range: Logistic Regression, LSTM, GRU, and RoBERTa.
- To improve on the accuracy reported by the base paper, both on our own data and, through cross-dataset validation, on the base paper's own dataset.
- To integrate LIME so that individual predictions from the best model can be explained at the level of contributing words.
- To position the results against two representative prior studies and to report the comparison honestly, including cases where an external implementation outperformed ours.

1.5 Key Contributions

This thesis makes four main contributions. First, it delivers a cyberbullying detector based on a fine-tuned RoBERTa model that reaches 92.33% accuracy on our combined dataset, a clear improvement over the 77.00% LSTM baseline of the base paper. Second, it adds LIME-based explainability to the detector, which neither of the two main comparison studies provides; this lets each decision be traced back to the words that drove it. Third, it validates the approach across datasets, applying the full pipeline to the base paper's own 46,017-comment Twitter data and still beating their reported result. Fourth, it presents a four-way comparison against a second recent study, using both our code and theirs on both datasets, and reports the outcome candidly, including an honest discussion of where

their implementation did better than ours.

Table 1.1. Overview of the thesis chapters

Chapter	**Main focus**
Introduction	Context, problem, objectives, and contributions
Literature Review	Prior work, the two base studies, and the research gap
Methodology	Data, preprocessing, the four models, LIME, and evaluation
Results and Discussion	Model performance, comparisons, LIME, and error analysis
Social Impact & Responsible Implementation	The social value, transparency, and responsible-deployment considerations of the system
Future Work	Directions for extending the study
Conclusion	Summary of findings and limitations

Table 1.1 lays out how the rest of the report is organised.

1.6 Scope of the Study

A few boundaries are worth stating at the outset. The classification task is binary throughout: every comment is treated as either bullying or not bullying. During data labelling we did record finer bullying sub-types, but these are not used by the final models and are kept only for possible later work. All text is in English. The explainability component is applied to the best-performing model, RoBERTa, since that is the model a deployed system would use; the recurrent baselines are included for comparison rather than for explanation. Real-time deployment, multilingual coverage, and image or video content are outside the scope of this thesis.

1.7 Significance of the Study

The significance of this work is easiest to see if we separate the two audiences it serves. For research, it closes a specific and named gap: it takes a published LSTM detector that reported no explanations, improves substantially on its accuracy with a transformer, verifies that improvement on more than one dataset, and adds the explainability the original lacked. That combination, an accuracy gain that survives cross-dataset testing together with per-prediction transparency, is not something the studies we reviewed offer together. For practice, the value is in trust. A moderation tool that can point to the words behind a

decision is far easier to deploy responsibly than one that simply returns a label, because its decisions can be reviewed, its mistakes can be understood, and an affected user can be given a reason rather than a verdict. The sarcasm case discussed later in the thesis is a small but concrete example of this second benefit: the explanation does not just describe the model, it exposes a weakness that pure accuracy would have hidden.

1.8 Thesis Organisation

The remainder of the report follows the usual academic order. Chapter 2 reviews related work and states the research gap precisely, giving particular attention to the two studies against which our results are compared. Chapter 3 describes the dataset, the preprocessing pipeline, the four models and their hyperparameters, the LIME methodology, and the evaluation setup. Chapter 4 presents the results, the cross-dataset and four-way comparisons, the LIME analysis, and an error analysis. Chapter 5 considers the social impact and responsible implementation of the system. Chapter 6 sets out the limitations and the directions for future work, and the final chapter, Chapter 7, draws the conclusions.

CHAPTER 2

LITERATURE REVIEW

Research on cyberbullying and abusive-language detection has grown quickly over the past decade, moving in step with the wider progress of natural language processing. This chapter first traces that progress from early feature-based classifiers to the transformer models used today, then turns to the smaller but growing literature on explainability. After that it reviews twelve representative studies in detail, each with a short account of what the authors did and how our work relates to it, and gathers them into a comprehensive summary table. The chapter closes by stating the research gap that this thesis sets out to fill.

2.1 Background

Automatic detection of harmful online content is usually cast as supervised classification over short texts such as tweets, comments, or forum posts [12]. Early efforts leaned on lexicons of offensive words and simple statistical features, which worked reasonably well on clear-cut cases but struggled with context, sarcasm, and the constant invention of new slang. As labelled datasets became available, the task matured into a standard text-classification benchmark, and researchers began to report the familiar metrics of accuracy, precision, recall, and F1. A recurring theme in this literature is that the definition of "bullying" is itself contested: some datasets label individual toxic messages, others require a pattern of repeated aggression, and this variation makes cross-study comparison harder than the headline numbers suggest [12].

2.2 Traditional Machine Learning Approaches

The classical approach represents each document as a sparse feature vector, most commonly using bag-of-words counts or Term Frequency–Inverse Document Frequency (TF–IDF) weights, sometimes extended with word or character n-grams. These vectors are then passed to a general-purpose classifier such as Logistic Regression, Support Vector Machine, Naive Bayes, or Random Forest. The appeal of this family is practical: the models train in seconds, they are cheap to run, and their behaviour is comparatively easy to inspect because a linear model assigns an explicit weight to each feature. These methods remain relevant because they provide a strong, honest baseline against which more complex models must justify their added cost, and because they are still competitive on smaller datasets where large neural models tend to overfit. In our work a TF–IDF Logistic Regression model plays exactly this baseline role.

2.3 Deep Learning Approaches

Neural networks changed the pipeline by learning dense word representations instead of relying on hand-built sparse features. Recurrent neural networks, and in particular the Long Short-Term Memory (LSTM) network of Hochreiter and Schmidhuber [16], process a sentence word by word while maintaining a memory of what came before, which lets them capture word order and longer-range dependencies that a bag-of-words model discards. The Gated Recurrent Unit (GRU) of Cho et al. [17] is a lighter variant that uses fewer gates and often trains faster while performing comparably. Both architectures became standard tools for abusive-language detection, and the base paper of this thesis belongs to this recurrent generation. Their main drawback is that they read a sentence in sequence and can lose information across long spans, and they must be trained from scratch on the task's own data, which limits them when that data is scarce.

2.4 Transformer-Based Language Models

The transformer architecture introduced by Vaswani et al. [13] replaced recurrence with self-attention, allowing every word to attend directly to every other word in the input and removing the sequential bottleneck. Building on this, BERT [14] showed that a large model pretrained on unlabelled text and then fine-tuned on a specific task could set new records across a wide range of NLP problems. RoBERTa [15] refined BERT's pretraining recipe, training longer on more data with larger batches and dropping the next-sentence-prediction objective, and consistently improved on BERT's results. Because these models

arrive already knowing a great deal about language, fine-tuning them on a modest cyberbullying dataset tends to outperform recurrent models trained from scratch. This is the reason RoBERTa is the model our work builds on.

2.5 Explainability in Text Classification

As models have grown more capable they have also grown more opaque, and this has driven interest in explainable artificial intelligence. For text classification, the most widely used technique is LIME, introduced by Ribeiro et al. [18]. LIME is model-agnostic: it treats the classifier as a black box, perturbs the input by removing words, observes how the predicted probability changes, and fits a simple interpretable model locally around the instance to estimate each word's contribution. A small number of recent cyberbullying and hate-speech studies have begun to apply such techniques [7, 11], but they remain the exception rather than the rule, and the two papers our work compares against most closely offer no explanation at all. This is the transparency our work supplies through LIME.

2.6 Review of Related Studies

This section reviews twelve studies that together map the current state of cyberbullying and abusive-language detection. For each, we summarise what the authors did and then note briefly how our work relates to it. The two studies marked as our primary comparisons are examined again, in experimental detail, in Chapter 4.

1. Esan et al. (2024) — the base paper. [1] The authors built an LSTM classifier on a Kaggle Twitter cyberbullying dataset of 47,692 tweets and reported 77.00% accuracy, with no interpretability component. This is the study our thesis directly extends. Our work improves on their accuracy by a wide margin, verifies the improvement on their own dataset, and adds the explainability their study lacks.

2. Ogunleye and Dharmaraj (2023) — primary comparison. [2] Working with a balanced English dataset of roughly 39,079 FormSpring and Twitter comments, they evaluated several models and reported RoBERTa as their best at around 87% accuracy and F1. Their public code and data allowed our four-way comparison in Chapter 4. Our work reaches higher accuracy on our own data and, like them, finds RoBERTa the strongest model, while additionally providing LIME explanations.

3. Alrowais et al. (2024) — RoBERTaNET. [3] This study combined a RoBERTa transformer with GloVe word embeddings and reported about 95% accuracy on a Kaggle

tweet dataset. It shows that hybrid representations can push transformer accuracy further, a direction we note for future work rather than adopt here; our work keeps a single fine-tuned RoBERTa for the sake of a clean, interpretable baseline.

4. Muneer et al. (2023) — stacking ensemble. [4] Published in Information, this work stacked a fine-tuned BERT with biLSTM and CNN components and reported 97.4% accuracy on a Twitter dataset and 90.97% on a combined Twitter–Facebook dataset. The very high accuracy comes at the cost of a complex, heavy pipeline and no explanation of individual decisions. Our work trades a little raw accuracy for a single, interpretable model whose decisions LIME can open up.

5. Roy and Mali (2022) — deep transfer learning. [5] Published in Complex & Intelligent Systems, this study applied convolutional networks and transfer learning and reached about 89% accuracy in its best case. It is an early demonstration of the value of transfer learning for this task, which is precisely the principle our fine-tuned RoBERTa relies on, applied here to text rather than images.

6. Ahmed, Mahmud and Biash et al. (2021) — the Bangla harassment dataset. [6] These authors collected and released 44,001 Bangla Facebook comments labelled across five categories (not-bully, troll, sexual, religious, and threat) and trained deep neural networks on them. This category scheme is closely related to the bullying sub-types we recorded during our own data collection, and the study is a natural reference point for the multilingual extension we propose in future work.

7. Karim et al. (2021) — DeepHateExplainer. [7] This work tackled hate-speech detection in Bengali and, unusually for its time, built in explainability, highlighting the terms behind a prediction through sensitivity analysis. It is one of the few prior studies to treat interpretability as a goal, and it supports the central argument of our thesis that explanation belongs inside a detection system rather than being bolted on afterwards.

8. Sihab-Us-Sakib et al. (2024) — Bengali multi-platform. [8] Using a Bengali dataset collected from Facebook, YouTube, and Instagram, the authors reported an XLM-RoBERTa model at 82.61%. Their multi-platform collection resembles our own use of YouTube, Reddit, and Facebook, though in a different language; our work outperforms their reported figure on our English data.

9. Azhar and Runa (2025) — transformer detection. [9] This recent study reported a RoBERTa model at 83.90% accuracy. It confirms the continuing dominance of RoBERTa-style models for this task, and our work exceeds its reported accuracy while adding the

explanation layer it does not include.

10. DeBERTa with Gated Broad Learning (2025). [10] This work paired a DeBERTa encoder with a gated broad learning system for English cyberbullying text, pursuing accuracy through a hybrid architecture. Like the ensemble studies, it points to architectural combination as a route to higher accuracy, which we list among our future directions.

11. LLM-based explainable detection in a streaming framework (2025). [11] This study used a large language model for cyberbullying detection within a stream-based machine-learning framework and, importantly, foregrounded explainability and trust. It is a sign that the field is beginning to take interpretability seriously, which is the same conviction that motivates our use of LIME, though we target a lighter and more reproducible setup.

12. A review of deep-learning cyberbullying detection (2023). [12] This survey, published in Future Internet, catalogues datasets, architectures, and evaluation practices across the field. It is useful for situating our work and confirms the two patterns we build on: the steady rise of transformer accuracy and the persistent shortage of explainability in deployed detectors.

2.7 Comprehensive Summary of Related Work

Table 2.1 draws the twelve studies together with our own, listing for each the best model, the dataset, the headline accuracy, and whether any explainability was provided. The final column is almost entirely "No," and that near-emptiness is the clearest single statement of the gap this thesis addresses. The table is laid out to be read on its own and is suitable for use as a presentation slide.

Table 2.1. Comprehensive summary of related work in cyberbullying and abusive-language detection

#	Study	Best Model	Dataset	Acc.	Expl.
1	Esan et al. (2024) [base]	LSTM	Twitter, 47,692	77.00%	No
2	Ogunleye & Dharmaraj (2023)	RoBERTa	FormSpring+Twitter	87.00%	No
3	Alrowais et al. (2024)	RoBERTa + GloVe	Kaggle tweets	95.00%	No
4	Muneer et al. (2023)	BERT+biLSTM+CNN	Twitter	97.40%	No
5	Roy & Mali (2022)	CNN + transfer learn.	Image/text media	89.00%	No
6	Ahmed, Mahmud & Biash (2021)	Deep neural net	Bangla FB, 44,001	–	No
7	Karim et al. (2021)	Transformer (Bengali)	Bengali hate speech	–	Yes
8	Sihab-Us-Sakib et al. (2024)	XLM-RoBERTa	Bengali FB/YT/IG	82.61%	No
9	Azhar & Runa (2025)	RoBERTa	English CB	83.90%	No
10	DeBERTa + GBLS (2025)	DeBERTa hybrid	English CB	–	No
11	LLM stream framework (2025)	Large lang. model	Social-media stream	–	Yes
12	Review (Future Internet, 2023)	Survey	Multiple	–	–
–	**Our Work**	**RoBERTa + LIME**	**YT/Reddit/FB + Jigsaw**	**92.33%**	**Yes**

2.8 Research Gap

Taken together, the reviewed literature shows steady gains in accuracy, driven most recently by pretrained transformers, alongside a persistent neglect of interpretability in the cyberbullying setting specifically. The base paper reports a recurrent model with no explanation of its decisions [1], and the main comparison paper, while stronger, is confined to English-only single-source data and likewise offers no explanations [2]. The handful of studies that do address explainability [7, 11] tend to work in other languages or heavier settings. The gap our work fills is therefore the combination of three things in one study: a transformer model that improves measurably on the base paper's accuracy, verified across more than one dataset; multi-platform data collected from YouTube, Reddit, and Facebook rather than a single source; and per-prediction explainability through LIME, which neither main comparison study provides.

2.9 Chapter Summary

The path traced through this chapter runs from feature-based classifiers, through recurrent networks, to pretrained transformers, and it is a path of steadily rising accuracy. Running alongside it, however, is a second and much thinner thread on explainability, one that the cyberbullying literature in particular has mostly left untouched. The twelve studies reviewed here, summarised in Table 2.1, sit at different points on the accuracy path but share, with only two exceptions, the same silence on interpretability. That intersection, strong transformer accuracy on one hand and an unmet need for explanation on the other, is where our work is positioned, and the methodology described in the next chapter is built to occupy exactly that space.

CHAPTER 3

METHODOLOGY

This chapter sets out how the study was carried out, from data collection through to the evaluation of the final models. It describes the two sources that make up the dataset, the preprocessing pipeline, the way the binary labels were prepared, the train/validation/test split, and the four models that were trained. It then explains how LIME was applied to the best model and, finally, which metrics and computing environment were used. The intention is that another researcher could reproduce the pipeline from this description together with the reported hyperparameters.

3.1 Overview of the Approach

The overall pipeline is straightforward and is applied identically to every model so that comparisons are fair. Raw comments are cleaned, deduplicated, and reduced to a single binary label; the cleaned dataset is shuffled and split once into training, validation, and test sets with a fixed random seed; each of the four models is trained on the training set and, for the neural models, monitored on the validation set; and every model is finally evaluated on the same held-out test set. LIME is then applied on top of the best model to explain individual predictions. Figure 3.1 summarises this flow.

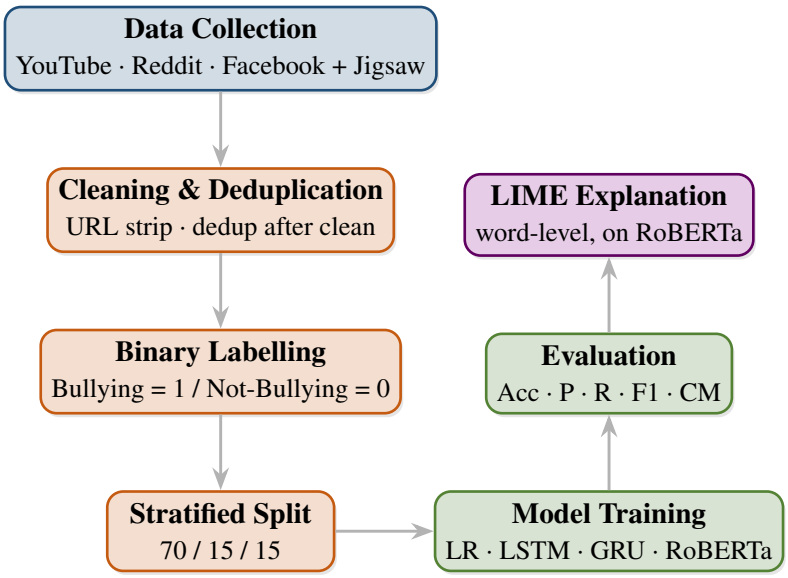


Figure 3.1. End-to-end pipeline: data collection, cleaning and deduplication, binary labelling, stratified splitting, training of the four models, evaluation, and LIME explanation on the best model.

3.2 Dataset

The dataset combines two sources into a single file, `CyberBullying Data.csv`, containing roughly 4,000 English comments. Each row holds the comment text, a binary label (Bullying or Not-Bullying), and, where applicable, a bullying sub-type that was recorded during labelling but is not used by the final models.

3.2.1 Self-Collected Data

The first source is a set of about 1,000 comments that we collected ourselves from three platforms: YouTube[1], Reddit[2], and Facebook[3]. Collecting from more than one platform was a deliberate choice, since it exposes the models to a wider range of writing styles, topics, and forms of hostility than any single source would. The raw collection was quality-filtered before labelling: exact and near duplicates were removed, spam and advertising were discarded, and very short or garbled entries that carried no usable signal were dropped. The surviving comments were then labelled manually. During this step we used a fine-grained scheme that recorded not only whether a comment was bullying but, for bullying cases, what kind it was; these sub-type labels are retained in the data but, as explained below, play no role in the final binary task.

[1] YouTube: `https://www.youtube.com`
[2] Reddit: `https://www.reddit.com`
[3] Facebook: `https://www.facebook.com`

3.2.2 Supplementary Data

The self-collected set alone is small for training transformer models, so it was supplemented with about 3,000 comments derived from the Jigsaw Toxic Comment Classification Challenge dataset [19], a large public corpus of Wikipedia talk-page comments released under a CC0 licence with categories such as toxic, severe toxic, obscene, threat, insult, and identity hate. We content-filtered this corpus and mapped its categories onto our own bullying sub-types, keeping only genuinely hostile comments and discarding neutral or merely topical statements that risked being mislabelled as bullying. This produced a supplementary set that is consistent in spirit with the self-collected data while adding the volume needed to train the deeper models.

3.2.3 Label Preparation and the Binary Reduction

The two sources were recorded with slightly different labelling schemes: the self-collected comments used a broader set of bullying sub-types, while the Jigsaw-derived comments were mapped onto a narrower one. This mismatch does not affect the study, because the final classification task is binary. Every comment from both sources was reduced to a single label, Bullying or Not-Bullying, regardless of its original sub-type. In other words, differing category granularity across the two sources had no bearing on the merge: each comment was mapped to the binary label and the sub-type information was simply set aside. In the code, the label is derived by lower-casing the original label field and mapping "bullying" to 1 and everything else to 0.

3.2.4 Dataset Statistics

After merging and cleaning, the dataset is close to balanced between the two classes, which is helpful because it means accuracy is a meaningful headline metric and no resampling was required. Table 3.1 summarises the class distribution and basic text statistics, and Table 3.2 shows the distribution of the recorded bullying sub-types among the bullying comments, included here for completeness even though the models do not use them.

Table 3.1. Dataset summary after cleaning and deduplication

Attribute	Value
Total comments	4,000
Not-Bullying	2,059
Bullying	1,941
Average length	22.6 words
Training split (70%)	2,800
Validation split (15%)	600
Test split (15%)	600

Table 3.2. Recorded bullying sub-types among the bullying comments (not used in the binary models)

Sub-type	Count
Troll	521
Religion	492
Sexual	399
Threat	366
Political	112
Racism/Ethnicity	51
Total bullying	1,941

3.2.5 Labelled Examples

To make the labelling concrete, Table 3.3 gives ten real comments drawn from the dataset, chosen so that every recorded category appears at least once. The examples show why the task is harder than a simple keyword search: the Not-Bullying rows include strongly worded but non-targeting comments, while the bullying rows span mockery (Troll), attacks on belief (Religion), sexual harassment (Sexual), explicit threats (Threat), political abuse (Political), and ethnic hostility (Racism/Ethnicity). The most explicit profanity has been lightly masked for presentation; the underlying data is unaltered.

Table 3.3. Representative labelled examples from the dataset, covering all six bullying sub-types

#	Comment	Label	Type
1	Show this video and Ronaldo's stats to him, now.	Not-Bullying	–
2	RELEASE ALL OF THE TRUMP/EPSTEIN FILES!!!	Not-Bullying	–
3	Man this girl sounds stupid	Bullying	Troll
4	Bolton is a clown	Bullying	Troll
5	Religion is primitive and a big waste of time.	Bullying	Religion
6	can't stop staring at ur body, u r so sexy	Bullying	Sexual
7	death to the infidel, whoever that may be	Bullying	Threat
8	So f*** off and die	Bullying	Threat
9	This is what happens when you elect a moron as president.	Bullying	Political
10	Mass immigration is destroying the West; import the third world, you become the third world.	Bullying	Racism/Ethnicity

3.3 Preprocessing

Text preprocessing was kept deliberately light, for a specific reason: transformer tokenizers are trained to handle raw text, and aggressive cleaning can remove signal that the model would otherwise use. The steps applied were therefore limited to what genuinely helps. URLs (both `http` links and `www` addresses) were stripped, and runs of whitespace were collapsed to single spaces with leading and trailing spaces trimmed. Rows with missing text were dropped, and empty comments left after cleaning were removed.

One ordering decision proved important. Deduplication was performed after text cleaning rather than before. Near-duplicate comments that differ only by a trailing URL or by irregular spacing look distinct in their raw form and survive a naive deduplication pass, but once cleaning has removed those surface differences they collapse to identical strings and can be caught. Running deduplication after cleaning removed a set of near-duplicates that would otherwise have leaked across the train/test boundary, and this measurably improved the measured accuracy of every model. For the classical and recurrent models the text was used as cleaned; for RoBERTa, the model's own tokenizer performed subword tokenisation, so no further manual normalisation was applied.

3.4 Training, Validation, and Test Splits

The cleaned dataset was shuffled once with a fixed random seed of 42 and then divided into three parts in a $70/15/15$ ratio: a training set, a validation set, and a test set. Each part has a distinct job. The training set is where the models actually learn their parameters. The validation set is held back during training and used to make decisions about the model without touching the test data, so for the neural models it is what the early-stopping criterion watches, and it is what tells us when to stop before the model starts to overfit. The test set is opened only once, at the very end, to give an unbiased estimate of how each finished model performs on comments it has never seen. Keeping these three roles separate is what stops information from the test set leaking into the training process and inflating the reported numbers. This three-way division is standard practice in supervised machine learning and is the arrangement recommended for a fair and reproducible evaluation [12]; the base paper and the main comparison study both rely on the same train/validation/test discipline [1, 2], which also makes our numbers directly comparable with theirs.

The split was stratified on the binary label, which means the roughly even Bullying/Not-Bullying balance of the full dataset is preserved in all three parts rather than being left to chance. Table 3.4 shows the resulting sizes and per-class counts. Crucially, the very same three subsets, produced by the same seed, are reused for every one of the four models, so that any difference in performance can be attributed to the models themselves and not to one model happening to get an easier slice of the data.

Table 3.4. Training, validation, and test split (stratified, seed $= 42$)

Split	Not-Bullying	Bullying	Total
Training (70%)	1,441	1,359	2,800
Validation (15%)	309	291	600
Test (15%)	309	291	600
Total	2,059	1,941	4,000

3.5 Feature Representation and Models

Four models were implemented, chosen to span the range from a classical linear baseline to a modern pretrained transformer. Using this spread makes it possible to see how much, if anything, the additional complexity of each step buys on this task.

3.5.1 Logistic Regression

The baseline is a TF–IDF Logistic Regression model. The cleaned text is converted to TF–IDF features using unigrams and bigrams with the vocabulary capped at the 10,000 most frequent terms. These features are passed to a Logistic Regression classifier with the regularisation strength set to $C = 1$ and a maximum of $1{,}000$ iterations. This model is fast, transparent, and provides an honest reference point for the neural models.

3.5.2 LSTM

The first deep model is an LSTM network. Text is tokenised with a Keras tokenizer restricted to the 10,000 most frequent words, with an explicit out-of-vocabulary token, and each sequence is padded or truncated to a length of 100 tokens. The network begins with a trainable embedding layer of dimension 128, followed by a stacked pair of LSTM layers with 128 and 64 units, a dropout layer with rate 0.3 to reduce overfitting, and a single sigmoid output unit for the binary decision. It is trained with the Adam optimiser and binary cross-entropy loss, in batches of 32, for up to 30 epochs, with early stopping on the validation loss (patience of 3 epochs) restoring the best weights.

3.5.3 GRU

The GRU model mirrors the LSTM exactly except that the two recurrent layers are Gated Recurrent Units of 128 and 64 units. Keeping the surrounding architecture, embedding size, sequence length, optimiser, batch size, dropout, and early-stopping policy identical means the LSTM and GRU results can be read as a direct comparison of the two recurrent cell types on this data.

3.5.4 RoBERTa

Our main model is a fine-tuned RoBERTa. We use the `roberta-base` model and its matching tokenizer, which encodes each comment into subword tokens with attention masks, truncating or padding to a maximum length of 128 tokens. A sequence-classification head with two output labels is placed on top of the pretrained encoder. The model is fine-tuned with the AdamW optimiser at a learning rate of 2×10^{-5}, using a linear learning-rate schedule with no warmup and gradient clipping at a norm of 1.0, in batches of 32, for up to 6 epochs. After each epoch the validation loss is checked; the best checkpoint is saved, and training stops early if the validation loss fails to improve for two consecutive epochs. The saved best checkpoint is the one evaluated on the test set.

3.5.5 Rationale for the Chosen Models

The four models were not picked at random; they were chosen to mark out a ladder of increasing capability so that each rung earns its place. Logistic Regression on TF–IDF features is the bottom rung, a fast linear model that tells us how far pure word-frequency information can carry the task on its own. The LSTM and GRU add the ability to read a sentence in order and so to react to phrasing rather than to isolated words, and comparing the two lets us see whether the particular recurrent cell makes much difference here. RoBERTa is the top rung, a large model that arrives already knowing a great deal about English from pretraining and then adapts that knowledge to our task. Laying the models out this way means the final results are not just a single number but a story about what each added capability buys, and it also keeps a strong, honest baseline in the picture so that the transformer's advantage has to be demonstrated rather than assumed.

3.6 Explainability with LIME

Explainability is provided through LIME [18], applied to the fine-tuned RoBERTa model. This section explains why LIME was chosen over the alternatives, how it works, and how it was configured in our experiments.

3.6.1 Why We Used LIME

A transformer such as RoBERTa is, from the user's point of view, a black box: it returns a label and a probability but no reason. For a cyberbullying detector that may lead to a comment being removed or a user being warned, that silence is a real problem, and it is the exact weakness we found in the base paper and the other studies reviewed in Chapter 2. We therefore wanted a method that could attach a human-readable reason to every individual prediction. LIME was chosen for four concrete reasons. First, it is **model-agnostic**: it treats the classifier purely as a function from text to probabilities, so the same tool works for RoBERTa, for the recurrent models, or for any future model without modification. Second, it is **local**: it explains one specific comment rather than the model as a whole, which is precisely what a moderator reviewing a single flagged comment needs. Third, its output is **word-level and intuitive**, a ranked list of the words that pushed the decision one way or the other, which a non-expert can read at a glance. Fourth, it is **lightweight and reproducible**, needing only repeated calls to the model rather than access to its gradients or internals, unlike heavier alternatives. We considered SHAP, which gives theoretically grounded global feature attributions, but for single-comment text explanations it is slower and its output is harder for a lay reviewer to interpret; LIME's per-word bars matched our

goal of a transparent, trustable decision far better.

3.6.2 How LIME Works

LIME explains one prediction at a time through a short, repeatable procedure, illustrated in Figure 3.2. First, it takes the comment to be explained and generates a large number of perturbed variants by randomly removing subsets of its words. Second, each variant is passed through RoBERTa to record the predicted probability of the Bullying class, so that LIME can observe how the prediction moves as different words are taken away. Third, it fits a simple weighted linear model over these perturbed samples, giving more weight to variants that stay close to the original comment. Fourth, the coefficients of that local linear model become the per-word importance scores: a positive score means the word pushed the prediction toward Bullying, a negative score means it pushed toward Not-Bullying, and the size of the score reflects how strong that push was.



Figure 3.2. How LIME produces a word-level explanation for a single comment.

3.6.3 LIME Configuration and Output

In our setup the two class names are Not-Bullying and Bullying, the prediction function returns softmax probabilities from RoBERTa in batches of 32 for efficiency, and each explanation is computed with 10 features and 500 perturbation samples. The output for any comment is three things at once: the predicted class, a confidence score, and a ranked, signed list of the words that mattered most. In the interactive tool this “important words” list is printed as a table before the bar chart is drawn; Table 3.5 reproduces it for the comment “Everyone hates you, just go away,” which RoBERTa classified as Bullying with 93.30% confidence. Every word here pushes toward Bullying, with hates and you carrying the most weight, which is exactly the explanation a human reviewer would give.

Table 3.5. LIME "important words" output for the comment "Everyone hates you, just go away"

Word	Weight	Direction
hates	+0.37	→ Bullying
you	+0.32	→ Bullying
away	+0.24	→ Bullying
Everyone	+0.21	→ Bullying
go	+0.12	→ Bullying
just	+0.06	→ Bullying

The value of this is simple but important: it lets a human see not just what the model decided but which words led it there, and, as the sarcasm example in the next chapter shows, that visibility is what turns an opaque mistake into a diagnosable one.

3.7 Evaluation Metrics

All models are evaluated on the same held-out test set using accuracy, precision, recall, and F1-score. Because the dataset is close to balanced, accuracy is a meaningful summary, but precision, recall, and F1 are reported alongside it to capture the trade-off between missing bullying comments and wrongly flagging harmless ones. Precision, recall, and F1 are computed with weighted averaging across the two classes. For a class, precision is the fraction of comments predicted as that class that truly belong to it, recall is the fraction of that class the model actually recovered, and the F1-score is their harmonic mean:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.1}$$

where TP, FP, and FN denote true positives, false positives, and false negatives respectively. In addition, a confusion matrix is reported for the best model so that its errors can be inspected directly.

3.8 Experimental Setup and System Specification

This section records the exact environment in which the experiments were run, both so that the results can be reproduced and so that the computational cost of each model is clear. All experiments were carried out in a cloud-based Google Colab notebook with GPU acceleration, which was necessary for fine-tuning RoBERTa in a reasonable time; the classical and recurrent models are light enough to run on the same environment com-

fortably.

3.8.1 Hardware Specification

Table 3.6 lists the hardware of the Colab runtime used for training and evaluation. The single GPU was the main enabler for the transformer: fine-tuning RoBERTa on a CPU alone would have taken many hours per epoch, whereas on the GPU each epoch completed in a few minutes.

Table 3.6. Hardware specification of the experimental environment

Component	Specification
Platform	Google Colab (cloud runtime)
GPU	NVIDIA Tesla T4, 16 GB GDDR6
GPU compute	CUDA-enabled, ∼2,560 CUDA cores
CPU	Intel Xeon, 2 vCPU @ 2.20 GHz
System RAM	∼13 GB
Disk	∼78 GB (temporary)

3.8.2 Software Specification

Table 3.7 lists the software stack. The classical baseline used scikit-learn [24], the recurrent models used TensorFlow/Keras [23], and RoBERTa was fine-tuned with the Hugging Face Transformers library [22] on top of PyTorch [21], while LIME was provided by the `lime` package [18].

Table 3.7. Software specification of the experimental environment

Software / Library	Role
Python 3	Programming language
PyTorch	Backend for RoBERTa fine-tuning
Hugging Face Transformers	RoBERTa model and tokenizer
TensorFlow / Keras	LSTM and GRU models
scikit-learn	TF–IDF, Logistic Regression, metrics
lime	Model explanations
Matplotlib / Seaborn	Plots and confusion matrices
pandas / NumPy	Data handling

To keep the results reproducible, a single random seed of 42 was set across Python's `random` module, NumPy, and PyTorch, and the same data split was reused for every model.

3.9 Chapter Summary

This chapter has described the full experimental method, and a few choices are worth restating because they carry through to the results. The dataset combines self-collected multi-platform comments with a supplementary set derived from a public corpus, and everything is reduced to a single binary label. Preprocessing is deliberately light, with the one non-obvious decision being to deduplicate after cleaning rather than before. The data is split once, stratified, with a fixed seed, and that same split is reused for all four models so the comparison is fair. The four models climb a ladder from a linear baseline to a fine-tuned transformer, and LIME sits on top of the best of them to explain individual predictions. Evaluation uses accuracy together with precision, recall, and F1, backed by confusion matrices. With the method fixed, the next chapter turns to what it produced.

CHAPTER 4

RESULTS AND DISCUSSION

This chapter reports what the experiments described in Chapter 3 actually produced. It opens with an overview of the dataset the models were trained on, then reports how the four models performed on the held-out test set and looks at each model's confusion matrix in turn. After that it sets our results against the two reference studies, first the base paper through cross-dataset validation and then the second study through a four-way comparison. The last part of the chapter is given over to the LIME analysis and a detailed error analysis, including the sarcasm case that the model gets wrong, before a short closing discussion.

4.1 Dataset Overview

Before turning to model performance it is worth looking at the data itself, since the shape of the dataset explains a good deal of what follows. Each row of the dataset holds three fields: the comment text, a binary label, and, for bullying comments only, a recorded sub-type. Representative labelled examples covering all six sub-types were already shown in Chapter 3 (Table 3.3), so here we focus on the overall statistics of the data rather than repeating individual rows.

After cleaning and deduplication the combined dataset holds 4,000 English comments, and the two classes are close to evenly matched: 2,059 comments are labelled Not-Bullying and 1,941 are labelled Bullying. In the encoded form used for training, the label is stored as an integer, where **0 stands for Not-Bullying and 1 stands for Bullying**;

this is the encoding shown on the horizontal axis of Figure 4.1, so the taller bar at 0 corresponds to the 2,059 non-bullying comments and the bar at 1 to the 1,941 bullying ones. The near-even split is convenient, because it means overall accuracy is a fair summary statistic and there was no need for resampling or class weighting to stop the models from simply favouring a majority class.

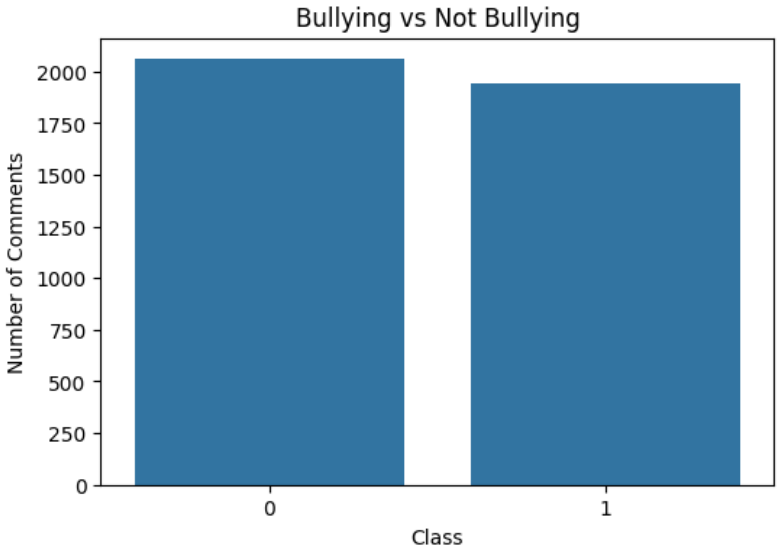


Figure 4.1. Class distribution of the combined dataset. Class 0 is Not-Bullying (2,059 comments) and class 1 is Bullying (1,941 comments).

The length of the comments matters too, because it affects how the sequence length for the recurrent models and the transformer was chosen. Figure 4.2 plots the distribution of comment length in words. The bulk of the comments are short, with the distribution peaking around ten to fifteen words and a long thin tail stretching out past a hundred words for a handful of unusually long comments. The average length works out to about 22.6 words. This is what justified capping the recurrent models at 100 tokens and RoBERTa at 128: those limits comfortably cover almost every comment while keeping the models efficient, since padding every short comment out to the length of the few very long ones would have wasted computation for no real benefit.

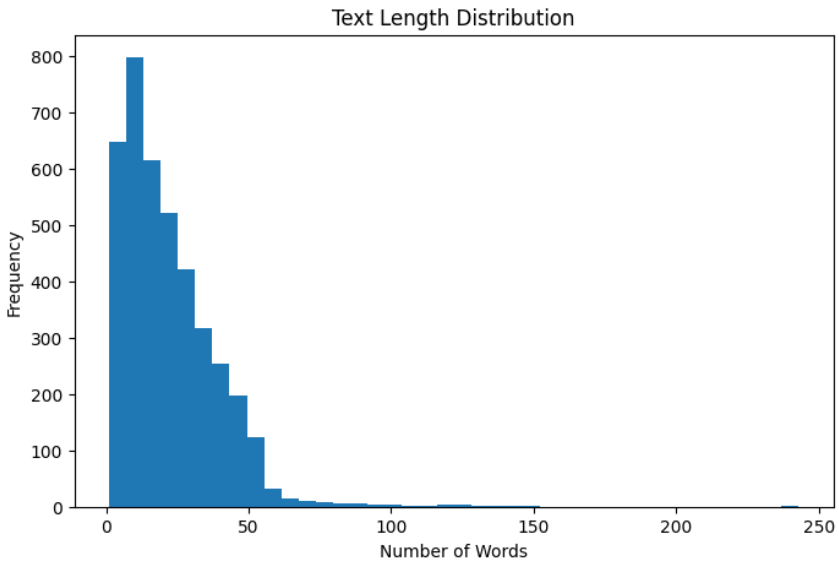


Figure 4.2. Distribution of comment length, measured in number of words. Most comments are short, with a long tail of longer ones.

4.2 Model Performance on the Combined Dataset

All four models were trained on the same 2,800-comment training set, tuned where relevant on the same 600-comment validation set, and finally scored on the same 600-comment test set. Reusing one split for every model is what makes the comparison fair, since any difference in the numbers then comes from the models rather than from the luck of the draw in how the data was divided. Table 4.1 collects the four metrics for each model.

Table 4.1. Performance of the four models on the combined dataset (test set, $n = 600$)

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression ($C = 1$)	83.33%	83.62%	83.33%	83.27%
GRU	85.33%	85.36%	85.33%	85.34%
LSTM	87.33%	87.33%	87.33%	87.33%
RoBERTa (our model)	**92.33%**	**92.36%**	**92.33%**	**92.33%**

The pattern is the one the literature would lead us to expect, and it is clean enough that it needs little interpretation. The Logistic Regression baseline reaches 83.33% accuracy, which is a genuinely respectable number for a linear model working on TF–IDF features, and it tells us that a large part of the signal in this task is carried by the words themselves rather than by their arrangement. The two recurrent models, which do model word order, improve on that: GRU reaches 85.33% and LSTM 87.33%. On this particular dataset the LSTM came out two points ahead of the GRU, though the two architectures are close

cousins and a gap of that size is not something to build an argument on. RoBERTa is the clear winner at 92.33%, five points clear of the best recurrent model, and its precision, recall, and F1 all sit within a hundredth of a point of its accuracy. That tightness across the four metrics, which holds for every model in the table, is itself reassuring: it means no model is quietly buying a good accuracy by sacrificing recall for precision or the other way round, which is exactly the kind of imbalance a near-even class split helps to avoid.

4.3 Confusion Matrix Comparison

Aggregate scores hide the shape of a model's mistakes, so it is worth looking at the confusion matrices on the 600-comment test set. Figure 4.3 places all four in a single view on a shared colour scale, which makes both the similarities and the differences between the models easy to read off at a glance.

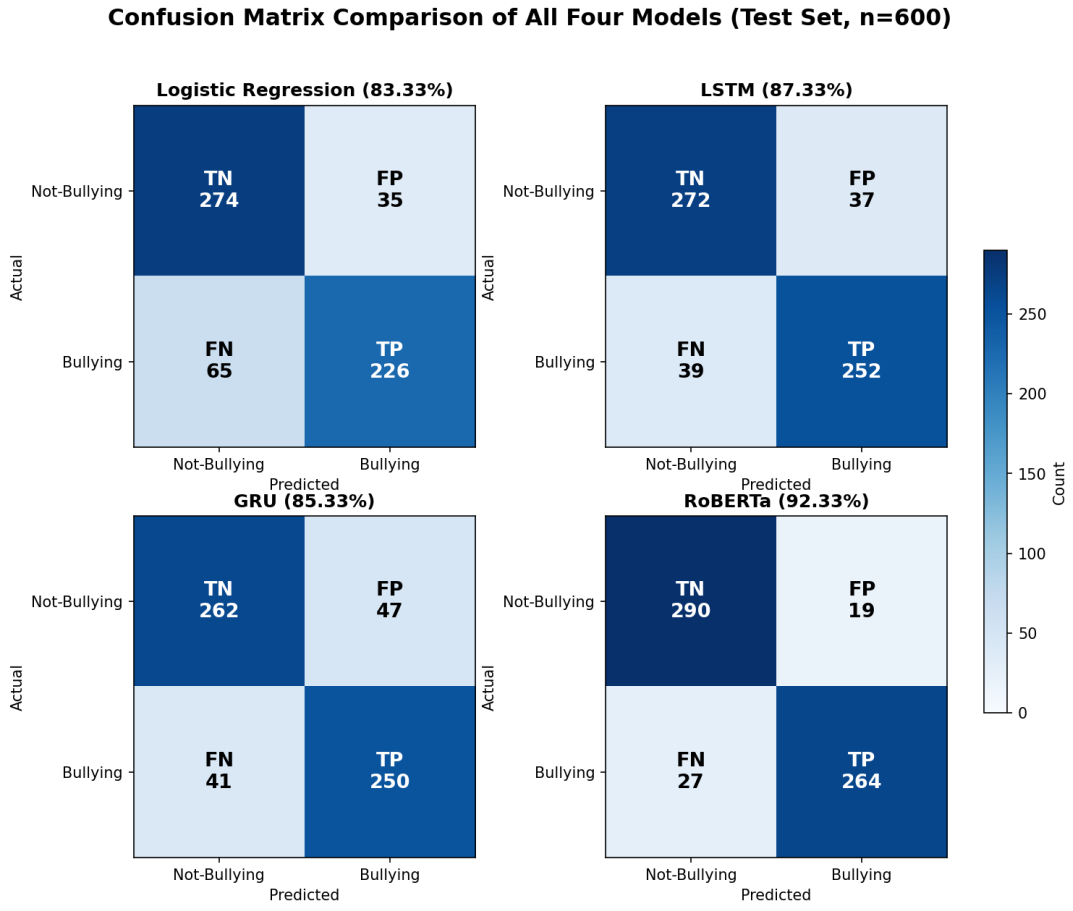


Figure 4.3. Confusion matrices of all four models on the test set ($n = 600$), drawn on a shared colour scale for direct comparison. TN = true negative, FP = false positive, FN = false negative, TP = true positive.

The **similarities** come first. Every model gets the majority of both classes right, so all

four matrices are darkest along the top-left to bottom-right diagonal, and none of them collapses into predicting a single class. The two recurrent models, LSTM and GRU, are almost indistinguishable from one another, which is unsurprising given how similar their architectures are.

The **differences** are where the story is. Logistic Regression stands out for its lopsided errors: it misses 65 bullying comments (the FN cell) against only 35 false positives, because a bag-of-words model struggles when hostility is carried by phrasing rather than by individual loaded words. Moving to the recurrent models, that imbalance shrinks, as LSTM cuts the misses to 39 and GRU to 41 by modelling word order, though both pay for it with a few more false positives (37 and 47 respectively). RoBERTa is visibly the strongest: its top-left cell is the darkest of all at 290 correct non-bullying comments with only 19 false positives, and it misses just 27 bullying comments. Its two error cells are not only the smallest but also the most balanced against each other, which is exactly the even-handed behaviour a moderation system wants. The 27 false negatives still slightly outnumber the 19 false positives, worth noting because a missed bullying comment is usually the costlier mistake; the error analysis later in this chapter looks at what those missed comments tend to look like.

Breaking RoBERTa's result down by class makes the balance clearer. Table 4.2 gives RoBERTa's per-class precision, recall, and F1-score, computed from the confusion matrix above. The two classes are handled almost symmetrically: the model is a little more precise on the Bullying class and a little better at recall on the Not-Bullying class, but every figure sits above 0.90, and neither class is being neglected in favour of the other. This even-handedness across classes is exactly what a moderation system needs, since a detector that was strong on one class and weak on the other would be of limited practical use however good its headline accuracy looked.

Table 4.2. Per-class classification report for RoBERTa on the test set

Class	Precision	Recall	F1-Score	Support
Not-Bullying	0.915	0.939	0.927	309
Bullying	0.933	0.907	0.920	291
Weighted average	0.924	0.923	0.923	600

4.4 Comparison with the Base Paper (Esan et al., 2024)

The base paper [1] reports an LSTM at 77.00% accuracy on a Kaggle Twitter dataset of 47,692 tweets [20]. Comparing that number directly with ours would not be fair, since

the two studies use different data, so we did the more demanding thing and ran our own pipeline on the base paper's own dataset. The data was obtained through a public mirror [20], the same cleaning pipeline was applied, and the models were evaluated on the cleaned 46,017-comment version.

Table 4.3. Same model (RoBERTa), different datasets

Dataset	RoBERTa Accuracy
Our dataset (∼4,000 comments)	92.33%
Esan et al. dataset (46,017 comments)	89.01%

Table 4.4. Same dataset (Esan et al., 46,017 comments), different models

Model	Accuracy	Precision	Recall	F1-Score
Esan et al. LSTM (paper-reported)	77.00%	–	–	–
Our Logistic Regression	86.63%	85.24%	86.63%	85.12%
Our LSTM	86.82%	87.71%	86.82%	87.19%
Our GRU	87.12%	86.51%	87.12%	86.74%
Our RoBERTa	**89.01%**	**88.95%**	**89.01%**	**88.98%**

Two things stand out from Tables 4.3 and 4.4. First, on the base paper's own data our RoBERTa reaches 89.01%, and even our recurrent and linear models land in the high 86% range, all comfortably above the base paper's reported 77.00%. The improvement over their LSTM is roughly twelve percentage points, and because it was measured on their own dataset it cannot be waved away as the result of easier data. Second, our RoBERTa scores lower on their larger, noisier Twitter data (89.01%) than on our own combined dataset (92.33%). That drop is a useful reminder that an accuracy figure only means something alongside the data it was measured on; the Twitter data is bigger and messier, and the model finds it harder.

4.5 Comparison with Ogunleye and Dharmaraj (2023)

The second comparison could be pushed further because the code and data of Ogunleye and Dharmaraj [2] are public, which let us cross the two datasets with the two codebases and produce the four settings in Table 4.5.

Table 4.5. Four-way comparison with Ogunleye and Dharmaraj (RoBERTa)

Setting	Dataset	Accuracy	F1-Score
Our data + our code	∼4,000	92.33%	92.33%
Our data + their code	∼4,000	**95.25%**	95.12%
Their Dataset 3 + our code	38,204	89.79%	89.79%
Their data + their published results	∼39,079	87.00%	87.00%

The single best RoBERTa result in the whole study, 95.25% accuracy, comes from running their code on our data. Our own implementation on our data reaches 92.33%, and our implementation on their Dataset 3 reaches 89.79%, both above their published 87.00%. Tables 4.6 and 4.7 give the full model breakdowns behind two of these settings.

Table 4.6. Full model breakdown — our data with the Ogunleye and Dharmaraj code

Model	Accuracy	Precision	Recall	F1-Score
TF-IDF + RF	83.25%	91.50%	72.16%	80.69%
TF-IDF + SVM	85.50%	88.64%	80.41%	84.32%
SBERT + RF	85.75%	87.43%	82.47%	84.88%
SBERT + SVM	87.00%	87.77%	85.05%	86.39%
XLNet	90.50%	91.05%	89.18%	90.10%
BERT	91.75%	92.59%	90.21%	91.38%
XLM-RoBERTa	93.00%	90.69%	95.36%	92.96%
RoBERTa	**95.25%**	**94.87%**	**95.36%**	**95.12%**

Table 4.7. Full model breakdown — their Dataset 3 with our code

Model	Accuracy	Precision	Recall	F1-Score
GRU	86.58%	86.75%	86.58%	86.58%
Logistic Regression	86.91%	87.01%	86.91%	86.91%
LSTM	87.04%	87.27%	87.04%	87.03%
RoBERTa	**89.79%**	**89.79%**	**89.79%**	**89.79%**

RoBERTa is the best model in every one of the four settings, and in the "our data + their code" setting it heads a strong field that includes XLM-RoBERTa at 93.00% and BERT at 91.75%. There is one result we want to be plain about rather than bury. Their code reached 95.25% on our data, higher than our own implementation's 92.33% on the very same data. Since the data is identical, the difference has to come from implementation and training details such as tokenisation choices, the learning-rate schedule, and other configuration. We treat that as a finding in its own right: it shows how much the training recipe can matter even when the model architecture and the data are held fixed, and it gives a concrete, evidence-backed direction for improvement, which we return to in the final chapter.

4.6 LIME Explainability Analysis

Accuracy tells us how often a model is right but nothing about why, and for a system that might drive a moderation decision the "why" is not optional. This is where LIME earns its place in the study. Applied to the fine-tuned RoBERTa model, it takes a single

comment, prints the predicted class and confidence, lists the Important Words with a signed weight and a direction arrow for each (a positive weight points toward Bullying, a negative weight toward Not-Bullying), and draws the same information as a bar chart. For each comment below we reproduce the exact Colab output followed by its bar chart. The comments were chosen because between them they show the technique working cleanly, leaning on a single decisive word, working under genuine uncertainty, and tripping over a tokenisation quirk; the sarcasm failure is examined separately in the error analysis.

4.6.1 A Clear, High-Confidence Case

The comment "Everyone hates you, just go away" was classified as Bullying with 93.30% confidence. The Colab output in Figure 4.4 lists the Important Words, and the bar chart in Figure 4.5 shows the same result: every word pushes toward Bullying, led by hates ($+0.3688$), you ($+0.3189$), and away ($+0.2388$), with Everyone adding a little more. This is the ideal case, a correct and confident prediction whose reasoning is exactly what a human would give and is easy to defend to a moderator or to the person on the receiving end.

```
Comment:
Everyone hates you, just go away

Binary Prediction: Bullying
Binary Confidence: 93.30%

Important Words:
------------------------------------------------
hates        +0.3688  -> Bullying
you          +0.3189  -> Bullying
away         +0.2388  -> Bullying
Everyone     +0.2127  -> Bullying
go           +0.1177  -> Bullying
just         +0.0595  -> Bullying
------------------------------------------------
```

Figure 4.4. Colab output for "Everyone hates you, just go away" (Bullying, 93.30% confidence).

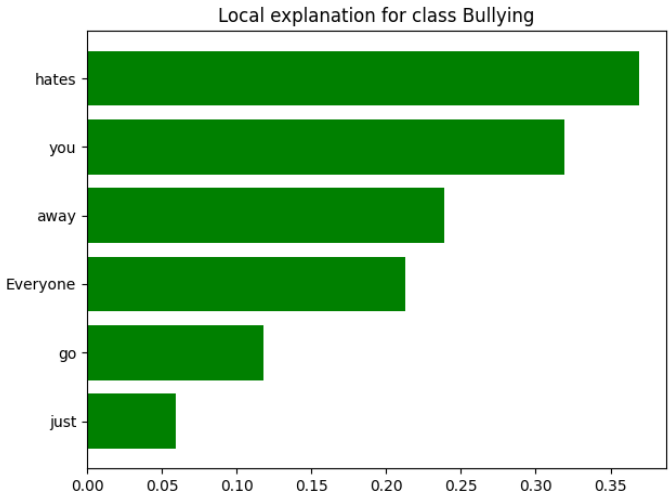


Figure 4.5. LIME bar chart for “Everyone hates you, just go away”.

4.6.2 A Strong Bullying Case Driven by One Word

The comment “Nobody asked for your stupid opinion” was classified as Bullying with very high confidence, 98.42%. The Colab output in Figure 4.6 and the bar chart in Figure 4.7 show that the decision rests almost entirely on the single word stupid ($+0.9156$), while every other word sits below 0.02 in either direction and barely matters. This is a helpful illustration of how the model latches onto a strongly insulting word when one is present, and of how LIME makes that dependence obvious rather than leaving it hidden inside the network.

```
Comment:
Nobody asked for your stupid opinion.

Binary Prediction: Bullying
Binary Confidence: 98.42%

Important Words:
--------------------------------------------------
stupid       +0.9156  -> Bullying
your         +0.0180  -> Bullying
asked        -0.0140  -> Not-Bullying
opinion      -0.0115  -> Not-Bullying
Nobody       -0.0053  -> Not-Bullying
for          +0.0020  -> Bullying
--------------------------------------------------
```

Figure 4.6. Colab output for “Nobody asked for your stupid opinion” (Bullying, 98.42% confidence).

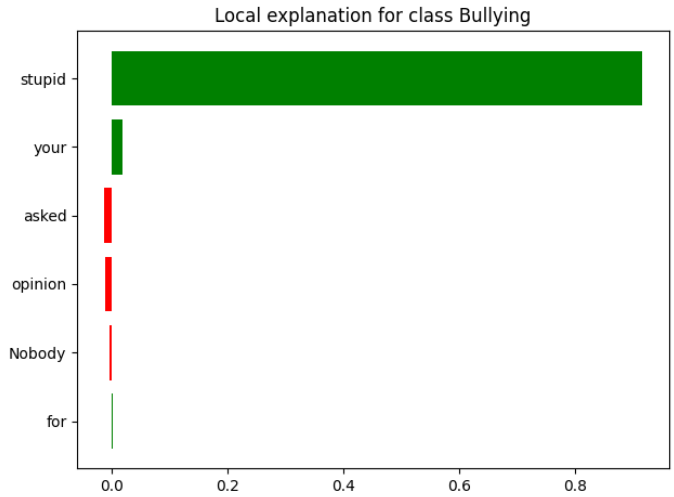


Figure 4.7. LIME bar chart for “Nobody asked for your stupid opinion”, dominated by stupid.

4.6.3 A Correct but Low-Confidence Case

The short comment “rubbish boy” was classified as Bullying, but only at 67.97% confidence. The Colab output in Figure 4.8 and the bar chart in Figure 4.9 show the weight spread across its two words, giving rubbish a weight of $+0.2321$ and boy a weight of $+0.1427$, both toward Bullying but neither large. The lower confidence is appropriate here: the comment is brief and its hostility is mild, so the model is right to hold back rather than commit strongly. This example matters because it shows the model expressing honest uncertainty on thin evidence instead of being falsely sure, and LIME makes that uncertainty visible.

```
Comment:
rubbish boy

Binary Prediction: Bullying
Binary Confidence: 67.97%

Important Words:
--------------------------------------------------
rubbish      +0.2321  -> Bullying
boy          +0.1427  -> Bullying
--------------------------------------------------
```

Figure 4.8. Colab output for “rubbish boy” (Bullying, 67.97% confidence).

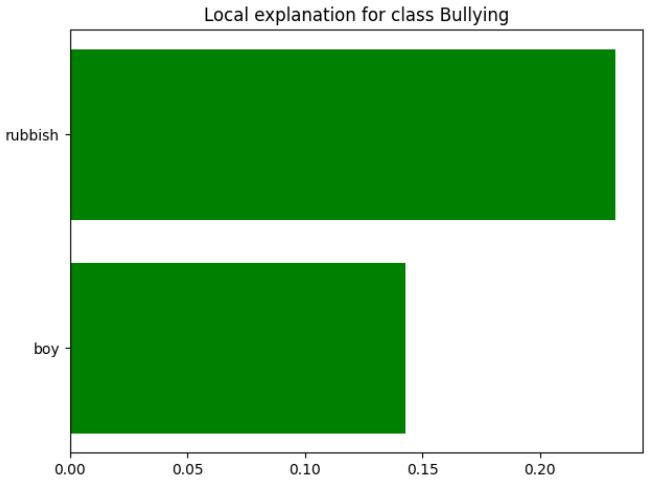


Figure 4.9. LIME bar chart for “rubbish boy”.

4.6.4 Distinguishing Disagreement from Bullying

The comment “I don’t agree with your opinion” was classified as Not-Bullying with 97.07% confidence, which is the right call, since disagreement is not the same as bullying and the model does not treat a plainly negative statement as hostile. In the Colab output in Figure 4.10 the individual weights are all small, none larger than about 0.03, which is consistent with a comment that carries no strongly hostile term; the strongest signals, opinion (-0.0269) and agree (-0.0211), both point toward Not-Bullying. The bar chart in Figure 4.11 also shows a small wrinkle: LIME splits “don’t” into the fragments don and t, an artefact of subword tokenisation that produces a slightly confusing per-token signal, since the two fragments even point in opposite directions. The prediction is still correct, but the quirk is a fair illustration of how word-level explanation can become murky when the tokenizer does not split text on word boundaries.

```
Comment:
I don't agree with your opinion

Binary Prediction: Not-Bullying
Binary Confidence: 97.07%

Important Words:
------------------------------------------------
opinion      -0.0269  -> Not-Bullying
agree        -0.0211  -> Not-Bullying
don          +0.0200  -> Bullying
with         -0.0063  -> Not-Bullying
t            +0.0061  -> Bullying
I            +0.0061  -> Bullying
your         +0.0060  -> Bullying
------------------------------------------------
```

Figure 4.10. Colab output for “I don’t agree with your opinion” (Not-Bullying, 97.07% confidence).

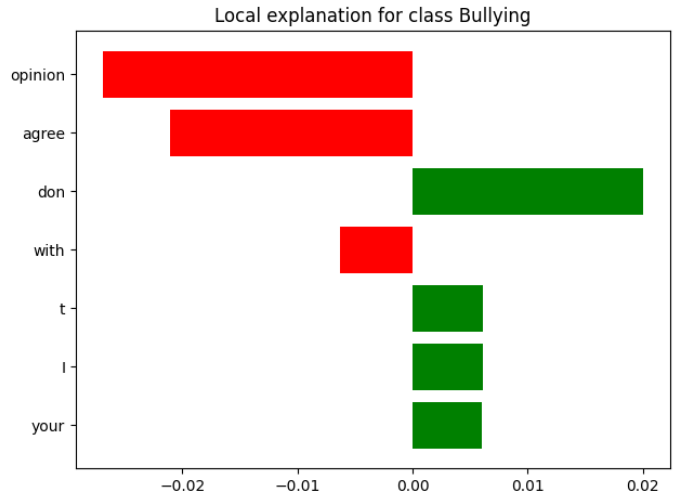


Figure 4.11. LIME bar chart for "I don't agree with your opinion". Note the split of "don't" into don and t.

4.7 Error Analysis

Looking across the comments the model gets wrong, the errors are not random; a few clear patterns run through them, and the most important of these is worth examining closely because it points to a real limitation rather than a stray mistake.

4.7.1 The Sarcasm Failure

The most instructive error in the whole study is a sarcastic comment. The sentence "Wow, you're really smart... said no one ever." was classified as Not-Bullying with 95.32% confidence, and that is wrong: the sentence is mockery, and the tag "said no one ever" flips its apparently positive wording into an insult. The Colab output in Figure 4.12 and the bar chart in Figure 4.13 show exactly how the model was fooled. The surface-positive words Wow (-0.0298), really (-0.0236), and said (-0.0200) pull hardest toward Not-Bullying, and although no ($+0.0186$), ever ($+0.0186$), smart ($+0.0177$), and you ($+0.0171$) each push a little toward Bullying, their weights are too small to flip the decision. The telltale phrase "said no one ever," which is what a human reads as the sarcastic reversal, is scattered across the fragments said, no, one, and ever that partly cancel one another out, so the phrase as a unit never registers. The model, in other words, reads the literal sentiment of the words and never grasps that the sentence as a whole means the opposite of what it appears to say.

```
Comment:
Wow, you're really smart... said no one ever.

Binary Prediction: Not-Bullying
Binary Confidence: 95.32%

Important Words:
------------------------------------------------
Wow          -0.0298  -> Not-Bullying
really       -0.0236  -> Not-Bullying
said         -0.0200  -> Not-Bullying
no           +0.0186  -> Bullying
ever         +0.0186  -> Bullying
smart        +0.0177  -> Bullying
you          +0.0171  -> Bullying
one          -0.0159  -> Not-Bullying
re           -0.0118  -> Not-Bullying
------------------------------------------------
```

Figure 4.12. Colab output for the misclassified sarcastic comment “Wow, you’re really smart... said no one ever.” (Not-Bullying, 95.32%, incorrect).

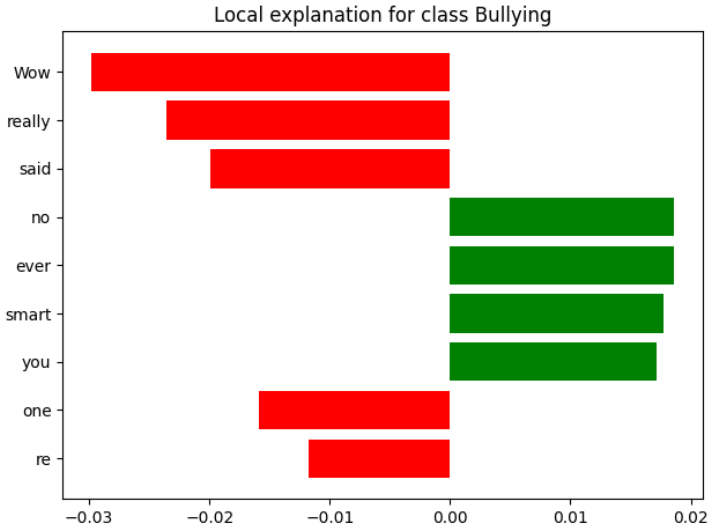


Figure 4.13. LIME bar chart for the misclassified sarcastic comment. The positive-sounding words Wow, really, and said (red) drive the wrong Not-Bullying prediction.

This is a textbook instance of a well-known hard problem in natural language processing. Sarcasm and irony carry their meaning in tone and in the relationship between parts of a sentence, not in any single hostile word, and a model that reasons largely from the words present will be misled whenever the words say one thing and the intent is the opposite. The error would be completely invisible from the confusion matrix or the accuracy figure, where it is just one misclassified comment among six hundred; it only becomes legible once the prediction is explained, which is precisely the argument for building explainability into the system in the first place.

4.7.2 Other Error Patterns

Two further patterns account for most of the remaining mistakes. Very short comments are a recurring source of trouble, simply because there is less evidence to weigh, and this is the same effect that gave the "rubbish boy" example its low confidence; when a comment is only two or three words long, a single ambiguous word can tip the decision either way. False positives, meanwhile, tend to come from strong but non-bullying language, where a comment is heated, negative, or profane without actually targeting a person, and the model occasionally mistakes that intensity for hostility. These patterns line up with RoBERTa's confusion matrix in Figure 4.3, where the errors are split fairly evenly between the two directions rather than piling up on one side.

4.8 Key Findings Summary

Before the closing discussion it is worth gathering the main results of this chapter into one place, since they are spread across several tables and comparisons. Table 4.8 does this, listing RoBERTa's accuracy in each of the settings tested along with the key point each one establishes.

Table 4.8. Summary of RoBERTa results across all experimental settings

Setting	Accuracy	What it shows
Our data + our code	92.33%	Our model, main result
Our data + their code	95.25%	Best overall; training details matter
Esan et al. data + our code	89.01%	Beats base paper on its own data
Their Dataset 3 + our code	89.79%	Beats their published result
Base paper (reported)	77.00%	The starting point we improve on
Their paper (reported)	87.00%	Second reference, also improved on

Read together, these rows make four points. RoBERTa was the best model in every single setting we tested, across our own data and both reference datasets. Our RoBERTa beats the base paper by a wide margin, at 92.33% on our data and 89.01% on the base paper's own data against their reported 77.00%. Our RoBERTa also beats the second study's published RoBERTa result of 87.00%, reaching 92.33% on our data and 89.79% on their own Dataset 3. And the one setting where an external implementation came out ahead, their code reaching 95.25% on our data, is not a counterexample to any of this but a pointer to how the training pipeline might be tightened further. Across all of it, the addition of LIME gives the detector an explainability layer that none of the compared studies offer.

4.9 Discussion

Taken together the results support the two central claims of the thesis. RoBERTa is the strongest model in every setting we tested, on our data, on the base paper's data, and on the second study's data, and it improves on the base paper's reported accuracy by a wide margin even when it is evaluated on the base paper's own dataset. Adding LIME then gives the detector something neither comparison study offers, namely the ability to explain each decision word by word, which does two jobs at once: it makes the model's correct decisions defensible, and, as the sarcasm case shows, it makes the model's weaknesses diagnosable. We have tried not to oversell the result. The gap between our RoBERTa implementation and the external implementation on identical data, 92.33% against 95.25%, shows there is real headroom left in our training pipeline, and the sarcasm failure marks a genuine limit on what a word-level model can currently do. Both of these carry directly into the conclusions and the future work that follow.

CHAPTER 5

SOCIAL IMPACT AND RESPONSIBLE IMPLEMENTATION

A cyberbullying detector is never just a piece of software. It sits between real people and the things they say to one another, and the decisions it makes can protect someone or, if used carelessly, silence them unfairly. Before drawing the thesis to a close, it is worth pausing on the human side of the work: who it could help, what it owes the people it judges, and how it should be used in practice.

5.1 Looking Out for the People Most at Risk

The clearest good this system can do is to reach the people who are hurt first. Online abuse falls hardest on teenagers and other vulnerable users, and for them the speed of a response matters as much as its accuracy. A detector that flags hostile comments automatically, around the clock and at the scale of millions of posts, can surface harm long before a human moderator would ever get to it. There is a quieter benefit too. Moderators spend their days reading the worst of what people write, and that takes a toll. By handling the obvious cases, a tool like ours lets them save their attention, and their resilience, for the genuinely difficult judgement calls that still need a person.

5.2 Decisions People Can Actually Question

Most detectors hand down a verdict and offer no reason, which is a hard thing to accept when the verdict is about your own words. This is where the explanations change the character of the system. Because LIME shows the words that drove each prediction, a decision stops being a black box and becomes something a moderator can review, defend, or overturn. Just as importantly, the person on the receiving end can be shown why their comment was flagged, rather than simply being told that it was. When the model gets it wrong, as it did on the sarcastic example we examined, that transparency is exactly what makes the mistake visible and correctable instead of hidden.

5.3 A Helper for Moderators, Not a Replacement

None of this works well if the system is handed the final say. We have been open about its blind spots, sarcasm and very short comments among them, and those blind spots are the reason it belongs beside a human rather than in place of one. Used as an assistant that recommends and explains while a person decides, it keeps responsibility where it should be. Applied in that spirit, an explainable detector can help build online spaces where abuse is caught early and dealt with consistently, which is good for ordinary users, easier on platforms, and a small step toward a safer and fairer internet.

CHAPTER 6

FUTURE WORK

This chapter takes the main limitations of the study and turns each one into a concrete plan. For each one we first name the issue in plain terms and then set out a concrete way forward, so that the path from the present study to a stronger one is clear. The directions are ordered roughly from the easiest and lowest-risk to the most ambitious.

6.1 Closing the Training-Pipeline Gap

The gap we noticed. On identical data, the reference implementation of Ogunleye and Dharmaraj [2] reached 95.25% with RoBERTa where our own implementation reached 92.33%. Our training configuration is therefore not yet optimal.

How we would close it. Because the data and the model architecture were the same in both cases, the three-point gap must lie in the training recipe rather than in the model itself. A systematic study of the tokenisation settings, the learning-rate schedule, the batch size, the number of fine-tuning epochs, and the handling of class weighting should recover most of that difference. This is the lowest-risk improvement available, since it needs no new data and no new model, only careful tuning and repeated validation; adopting the specific choices in the reference code and ablating them one at a time would show exactly which detail matters most.

6.2 Making the Model Sarcasm-Aware

Where the model slips. The clearest failure in our analysis was the sarcastic comment that RoBERTa confidently misread, because its hostility was carried by tone rather than by any hostile word.

Teaching it to read the tone. Sarcasm and irony are a known hard problem in NLP, but several routes could reduce these errors. Auxiliary training data explicitly labelled for irony would teach the model what tonal reversal looks like; a model pretrained on data rich in sarcasm, or a dedicated sarcasm-detection module whose output is fed to the classifier, could supply the missing signal; and context beyond the single comment, such as the thread it replies to, would often make the intent clear. Even a partial improvement here would remove one of the most visible weaknesses of the current system.

6.3 Restoring Fine-Grained Bullying Types

What we left on the shelf. The binary framing means the sub-type labels recorded during data collection (troll, religious, sexual, threat, political, and racial/ethnic) are not used by the final models, so information we actually captured is left unused.

Putting the detail back to work. These labels still exist in the dataset, so the extension is mostly a modelling task rather than a new annotation effort. The detector could be extended into a second, fine-grained stage that runs after the binary decision, framed either as a multi-class problem (one bullying type per comment) or as a multi-label one (a comment may be both, say, religious and threatening). This would turn the system from a simple "bullying or not" flag into one that also reports the kind of bullying, which is far more useful to a moderator deciding how to respond.

6.4 Raising Accuracy with Richer Representations and Ensembles

Room left at the top. A single fine-tuned RoBERTa, while strong, leaves some accuracy on the table compared with the heavier systems reported in the literature.

Two ways to climb higher. Two routes are promising. The first is hybrid representations: related work has combined RoBERTa with GloVe features and reported gains on similar data [3], and a similar combination could be tried here. The second is ensem-

bling: stacking several transformers, such as RoBERTa with BERT and XLM-RoBERTa, has repeatedly raised accuracy on comparable tasks [4], and the strong individual results of these models in our four-way comparison suggest an ensemble would likely beat any single member. The trade-off, which future work must weigh, is that both routes make the system heavier and its explanations harder to produce.

6.5 Scaling Up the Dataset

A modest sample. Our dataset of about 4,000 comments is modest next to benchmarks such as the base paper's 47,692 tweets, which limits how far the heavier models can be pushed and how confidently the results generalise.

Growing the corpus. The self-collected corpus can be expanded well beyond its current size by drawing on more YouTube, Reddit, and Facebook sources, ideally with multiple annotators and a measured inter-annotator agreement to keep label quality high. A larger corpus would let the transformer models train without the overfitting risk that a few thousand comments always carries, and would make robustness testing at scale meaningful.

6.6 Extending to Multiple Languages

English only, for now. The present study is English-only, yet the platforms it draws from are used heavily in multilingual settings, and online hostility frequently mixes languages within a single comment.

Going multilingual. The most ambitious and, to us, the most interesting direction is a multilingual detector covering **Bangla and English**, since the two are routinely mixed in comments from Bangladeshi users, with **Hindi** added as a third language where suitable data can be gathered. This is also the natural place to bring back the fine-grained types discussed above, so that the system would predict, across all three languages, both whether a comment is bullying and what kind it is, carrying a label and a type together. Code-mixed text, where two languages appear in one sentence, is a known challenge for tokenisers, so a multilingual transformer such as XLM-RoBERTa would be the sensible starting point, and the Bangla harassment dataset of Ahmed, Mahmud and Biash [6] together with the multilingual studies of Sihab-Us-Sakib et al. [8] and Azhar and Runa [9] would provide ready points of comparison. Combining three languages, fine-grained types, and LIME explanations in one system would be a substantial step beyond the present work and a strong basis for a follow-up paper.

CHAPTER 7

CONCLUSION

This thesis set out to build a cyberbullying detector that is both accurate and able to explain its decisions, and both aims were met. On a combined dataset of roughly 4,000 English comments collected from YouTube, Reddit, and Facebook and supplemented from the Jigsaw corpus, a fine-tuned RoBERTa model reached 92.33% accuracy, clearly ahead of the Logistic Regression (83.33%), GRU (85.33%), and LSTM (87.33%) baselines trained under the same conditions. RoBERTa stayed the strongest model in every setting we tested: on the base paper's own 46,017-comment Twitter data it reached 89.01%, far above their reported 77.00%, and in the four-way comparison against the second study it again led throughout. The central contribution, which neither comparison study offers, is that pairing the model with LIME turned every prediction into an inspectable decision, showing the exact words behind it.

The broader lesson is that accuracy and transparency need not be traded against each other: the same model that performed best is also the one whose decisions LIME could open up word by word, at no cost to accuracy. For a task where a wrong decision affects a real person, that combination matters more than a fractional gain on a leaderboard. We have also been honest about where the work can improve, most notably the training gap behind the external implementation's higher score and the model's weakness on sarcasm, and the preceding chapter turns each of these into a concrete direction for future work. Used responsibly, with a human kept in the loop, an explainable detector of this kind can make online moderation both more effective and more fair.

BIBLIOGRAPHY

[1] O. Abodunrin, A. Esan, A. Sobowale, T. Adebiyi, and A. Soladoye, "LSTM-Based Model for Cyberbully Detection," LAUTECH Journal of Engineering and Technology, vol. 18, no. 1, pp. 181–191, 2024.

[2] B. Ogunleye and B. Dharmaraj, "The Use of a Large Language Model for Cyberbullying Detection," Analytics, vol. 2, no. 3, pp. 694–707, 2023, doi: 10.3390/analytics2030038.

[3] F. Alrowais, "RoBERTaNET: Enhanced RoBERTa Transformer Based Model for Cyberbullying Detection with GloVe Features," IEEE Access, vol. 12, 2024.

[4] A. Muneer, A. Alwadain, M. G. Ragab, and A. Alqushaibi, "Cyberbullying Detection on Social Media Using Stacking Ensemble Learning and Enhanced BERT," Information, vol. 14, no. 8, art. 467, 2023, doi: 10.3390/info14080467.

[5] P. K. Roy and F. U. Mali, "Cyberbullying detection using deep transfer learning," Complex & Intelligent Systems, vol. 8, no. 4, pp. 5449–5467, 2022, doi: 10.1007/s40747-022-00772-z.

[6] M. F. Ahmed, Z. Mahmud, Z. T. Biash, A. A. N. Ryen, A. Hossain, and F. B. Ashraf, "Cyberbullying Detection Using Deep Neural Network from Social Media Comments in Bangla Language," arXiv preprint arXiv:2106.04506, 2021.

[7] M. R. Karim, S. K. Dey, T. Islam, et al., "DeepHateExplainer: Explainable Hate Speech Detection in Under-resourced Bengali Language," in IEEE International Conference on Data Science and Advanced Analytics (DSAA), 2021.

[8] S. Sihab-Us-Sakib, M. R. Rahman, M. S. A. Forhad, and M. A. Aziz, "Cyberbullying detection of resource constrained language from social media using transformer-based approach," Natural Language Processing Journal, vol. 8, art. 100104, 2024, doi: 10.1016/j.nlp.2024.100104.

[9] M. A. H. B. Azhar and A. A. Runa, "Enhancing Cyberbullying Detection with RoBERTa: A Transformer-Based Approach," in Proc. 16th Int. Conf. on Global Security, Safety & Sustainability (ICGS3), 2025.

[10] A. Hassan et al., "A Hybrid DeBERTa and Gated Broad Learning System for Cyberbullying Detection in English Text," arXiv preprint arXiv:2506.16052, 2025.

[11] Á. Fernández et al., "Promoting Security and Trust on Social Networks: Explainable Cyberbullying Detection Using Large Language Models in a Stream-Based Machine Learning Framework," arXiv preprint arXiv:2505.03746, 2025.

[12] M. T. Hasan, M. A. E. Hossain, M. S. H. Mukta, A. Akter, M. Ahmed, and S. Islam, "A Review on Deep-Learning-Based Cyberbullying Detection," Future Internet, vol. 15, no. 5, art. 179, 2023, doi: 10.3390/fi15050179.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in Advances in Neural Information Processing Systems, vol. 30, 2017.

[14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proceedings of NAACL-HLT, pp. 4171–4186, 2019.

[15] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv preprint arXiv:1907.11692, 2019.

[16] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[17] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," in Proceedings of EMNLP, pp. 1724–1734, 2014.

[18] M. T. Ribeiro, S. Singh, and C. Guestrin, " 'Why Should I Trust You?': Explaining the Predictions of Any Classifier," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1135–1144, 2016.

[19] Jigsaw and Conversation AI, "Toxic Comment Classification Challenge," Kaggle, 2018. [Online]. Available: https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge

[20] J. Wang, K. Fu, and C.-T. Lu, "Cyberbullying Classification Dataset (47,692 tweets)," Kaggle / GitHub mirror, 2020. [Online]. Available: https://github.com/leoAshu/cyberbullying-classification

[21] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems, vol. 32, 2019.

[22] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in Proceedings of EMNLP: System Demonstrations, pp. 38–45, 2020.

[23] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems," 2016. [Online]. Available: https://www.tensorflow.org

[24] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

Source Code

Code 1 — Install & Import

```
!pip install -q transformers datasets lime accelerate

import pandas as pd
import numpy as np
import re
import os
import random
import time

import torch
from torch.utils.data import Dataset, DataLoader

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    classification_report,
    confusion_matrix
)

import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, GRU, Dense, Dropout

from transformers import (
    RobertaTokenizer,
    RobertaForSequenceClassification,
    get_linear_schedule_with_warmup
)

from lime.lime_text import LimeTextExplainer
```

Code 2 — GPU & Seed

```
import random
import numpy as np
import torch

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Device:", device)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
else:
```

```
    print("GPU not available")
```

Code 3 — Dataset Load

```
df = pd.read_csv("CyberBullyingData.csv")

print("Dataset Shape:", df.shape)
print("Columns:", df.columns.tolist())

display(df.head())
```

Code 4 — Dataset Inspection

```
print("Total Rows:", len(df))

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Rows:", df.duplicated().sum())

print("\nLabel Distribution:")
print(df["Label"].value_counts())

print("\nType Distribution:")
print(df["Types"].value_counts(dropna=False))
```

Code 5 — Cleaning & Preprocessing

```
import re

df = df.dropna(subset=["Text"]).copy()
df["Text"] = df["Text"].astype(str)

def clean_text(text):
    text = str(text)
    text = re.sub(r"http\S+|www\.\S+", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df["clean_text"] = df["Text"].apply(clean_text)

df = df.drop_duplicates(subset=["clean_text"]).reset_index(drop=True)
df = df[df["clean_text"].str.strip() != ""].reset_index(drop=True)

df["binary_label"] = (
    df["Label"]
    .str.strip()
    .str.lower()
    .eq("bullying")
    .astype(int)
)

# Keep the bullying sub-type (Troll / Religion / Sexual / Threat / Political / Racism)
# so we can optionally filter by topic later. Not-Bullying rows get an empty topic.
df["Types"] = df["Types"].fillna("").astype(str).str.strip()

df = df[["clean_text", "binary_label", "Types"]].rename(
    columns={"clean_text": "text", "binary_label": "label", "Types": "topic"}
)

print("Final Dataset:", len(df))
print(df["label"].value_counts())
print(df.head())
```

Code 6 — Code Cell 6

```
# ============================================================
# Topic Filter (optional) - select related bullying topics
# ============================================================
# Available topics: Troll, Religion, Sexual, Threat, Political, Racism/Ethnicity
#
# SELECTED_TOPICS = None  ->  use ALL topics (full dataset, default)
# SELECTED_TOPICS = ["Religion", "Troll"]  ->  keep all Not-Bullying comments
#                                              + only these bullying topics
SELECTED_TOPICS = None

print("Topic counts (Bullying rows only):")
print(df[df["label"] == 1]["topic"].value_counts())

if SELECTED_TOPICS:
    keep = (df["label"] == 0) | (df["topic"].isin(SELECTED_TOPICS))
    df = df[keep].reset_index(drop=True)
    print("\nSelected topics:", SELECTED_TOPICS)
    print("Rows after topic filter:", len(df))
    print(df["label"].value_counts())
else:
    print("\nUsing ALL topics - full dataset:", len(df))
```

Code 7 — Class Distribution

```
import matplotlib.pyplot as plt
import seaborn as sns

print(
    df["label"]
    .value_counts()
)

plt.figure(figsize=(6, 4))

sns.countplot(
    data=df,
    x="label"
)

plt.title(
    "Bullying vs Not Bullying"
)

plt.xlabel("Class")
plt.ylabel("Number of Comments")

plt.show()
```

Code 8 — Text Length

```
df["text_length"] = (
    df["text"]
    .str.split()
    .str.len()
)

print(
    df["text_length"].describe()
)

plt.figure(figsize=(8, 5))

plt.hist(
    df["text_length"],
    bins=40
)

plt.title(
    "Text Length Distribution"
)

plt.xlabel("Number of Words")
plt.ylabel("Frequency")

plt.show()
```

Code 9 — Code Cell 9

```
from sklearn.model_selection import train_test_split

X = df["text"].values
y = df["label"].values

X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=SEED,
    stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=SEED,
    stratify=y_temp
)

print("Train:", len(X_train))
print("Validation:", len(X_val))
print("Test:", len(X_test))
```

Code 10 — Baseline 1 — Logistic Regression

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
import time

tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1, 2))
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

lr_model = LogisticRegression(C=1, max_iter=1000, random_state=SEED)

start = time.time()
lr_model.fit(X_train_tfidf, y_train)
lr_preds = lr_model.predict(X_test_tfidf)
lr_time = time.time() - start

lr_acc = accuracy_score(y_test, lr_preds)
lr_prec, lr_rec, lr_f1, _ = precision_recall_fscore_support(y_test, lr_preds, average="
    weighted")

print("Logistic Regression (tuned, C=1)")
print("Accuracy :", round(lr_acc, 4))
print("Precision:", round(lr_prec, 4))
print("Recall   :", round(lr_rec, 4))
print("F1 Score :", round(lr_f1, 4))
```

Code 11 — Baseline 2 — LSTM

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

MAX_WORDS = 10000
MAX_LEN = 100

keras_tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token="<OOV>")
keras_tokenizer.fit_on_texts(X_train)

X_train_seq = pad_sequences(keras_tokenizer.texts_to_sequences(X_train), maxlen=MAX_LEN)
X_val_seq = pad_sequences(keras_tokenizer.texts_to_sequences(X_val), maxlen=MAX_LEN)
X_test_seq = pad_sequences(keras_tokenizer.texts_to_sequences(X_test), maxlen=MAX_LEN)

lstm_model = Sequential([
    Embedding(MAX_WORDS, 128, input_length=MAX_LEN),
    LSTM(128, return_sequences=True),
    LSTM(64),
    Dropout(0.3),
    Dense(1, activation="sigmoid")
])
lstm_model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

early_stop = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)
checkpoint = ModelCheckpoint('best_lstm.h5', monitor='val_loss', save_best_only=True)

lstm_history = lstm_model.fit(
    X_train_seq, y_train,
    validation_data=(X_val_seq, y_val),
    epochs=30,
    batch_size=32,
    callbacks=[early_stop, checkpoint],
    verbose=1
```

```
)

lstm_probs = lstm_model.predict(X_test_seq, verbose=0).flatten()
lstm_preds = (lstm_probs >= 0.5).astype(int)

lstm_acc = accuracy_score(y_test, lstm_preds)
lstm_prec, lstm_rec, lstm_f1, _ = precision_recall_fscore_support(y_test, lstm_preds,
    average="weighted")

print("\nLSTM (tuned, stopped at epoch", len(lstm_history.history['loss']), ")")
print("Accuracy :", round(lstm_acc, 4))
print("Precision:", round(lstm_prec, 4))
print("Recall   :", round(lstm_rec, 4))
print("F1 Score :", round(lstm_f1, 4))
```

Code 12 — Baseline 3 — GRU

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GRU, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

gru_model = Sequential([
    Embedding(MAX_WORDS, 128, input_length=MAX_LEN),
    GRU(128, return_sequences=True),
    GRU(64),
    Dropout(0.3),
    Dense(1, activation="sigmoid")
])
gru_model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

early_stop_gru = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)
checkpoint_gru = ModelCheckpoint('best_gru.h5', monitor='val_loss', save_best_only=True)

gru_history = gru_model.fit(
    X_train_seq, y_train,
    validation_data=(X_val_seq, y_val),
    epochs=30,
    batch_size=32,
    callbacks=[early_stop_gru, checkpoint_gru],
    verbose=1
)

gru_probs = gru_model.predict(X_test_seq, verbose=0).flatten()
gru_preds = (gru_probs >= 0.5).astype(int)

gru_acc = accuracy_score(y_test, gru_preds)
gru_prec, gru_rec, gru_f1, _ = precision_recall_fscore_support(y_test, gru_preds, average
    ="weighted")

print("\nGRU (tuned, stopped at epoch", len(gru_history.history['loss']), ")")
print("Accuracy :", round(gru_acc, 4))
print("Precision:", round(gru_prec, 4))
print("Recall   :", round(gru_rec, 4))
print("F1 Score :", round(gru_f1, 4))
```

Code 13 — Main Model — RoBERTa

```
from transformers import RobertaTokenizer

MODEL_NAME = "roberta-base"

roberta_tokenizer = (
    RobertaTokenizer
    .from_pretrained(MODEL_NAME)
)

print("RoBERTa tokenizer loaded.")
```

Code 14 — Dataset & DataLoader

```
from torch.utils.data import Dataset, DataLoader

class BullyingDataset(Dataset):

    def __init__(
        self,
        texts,
        labels,
        tokenizer,
        max_len=128
    ):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):

        encoding = self.tokenizer(
            str(self.texts[idx]),
            truncation=True,
            padding="max_length",
            max_length=self.max_len,
            return_tensors="pt"
        )

        return {
            "input_ids":
                encoding["input_ids"].squeeze(0),

            "attention_mask":
                encoding["attention_mask"].squeeze(0),

            "labels":
                torch.tensor(
                    self.labels[idx],
                    dtype=torch.long
                )
        }


train_dataset = BullyingDataset(
    X_train,
    y_train,
    roberta_tokenizer
)

val_dataset = BullyingDataset(
```

```
    X_val,
    y_val,
    roberta_tokenizer
)

test_dataset = BullyingDataset(
    X_test,
    y_test,
    roberta_tokenizer
)


train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=32
)

test_loader = DataLoader(
    test_dataset,
    batch_size=32
)

print("DataLoaders ready.")
```

Code 15 — RoBERTa Training

```
from transformers import RobertaForSequenceClassification,
    get_linear_schedule_with_warmup
from sklearn.metrics import accuracy_score
import torch
import time

roberta_model = RobertaForSequenceClassification.from_pretrained(MODEL_NAME, num_labels
    =2).to(device)
optimizer = torch.optim.AdamW(roberta_model.parameters(), lr=2e-5)

EPOCHS = 6
PATIENCE = 2

total_steps = len(train_loader) * EPOCHS
scheduler = get_linear_schedule_with_warmup(optimizer, num_warmup_steps=0,
    num_training_steps=total_steps)

best_val_loss = float("inf")
epochs_no_improve = 0

print("Starting RoBERTa training (lr=2e-5)...")

for epoch in range(EPOCHS):
    start_time = time.time()
    roberta_model.train()
    total_train_loss = 0
    for step, batch in enumerate(train_loader):
        optimizer.zero_grad()
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)
        outputs = roberta_model(input_ids=input_ids, attention_mask=attention_mask,
    labels=labels)
        loss = outputs.loss
```

```
        loss.backward()
        torch.nn.utils.clip_grad_norm_(roberta_model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()
        total_train_loss += loss.item()
        if (step + 1) % 25 == 0 or (step + 1) == len(train_loader):
            print(f"Epoch {epoch+1}/{EPOCHS} | Step {step+1}/{len(train_loader)} | Loss:
    {loss.item():.4f}")

    train_loss = total_train_loss / len(train_loader)

    roberta_model.eval()
    total_val_loss = 0
    val_preds, val_labels = [], []
    with torch.no_grad():
        for batch in val_loader:
            input_ids = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            labels = batch["labels"].to(device)
            outputs = roberta_model(input_ids=input_ids, attention_mask=attention_mask,
    labels=labels)
            total_val_loss += outputs.loss.item()
            val_preds.extend(torch.argmax(outputs.logits, dim=1).cpu().numpy())
            val_labels.extend(labels.cpu().numpy())

    val_loss = total_val_loss / len(val_loader)
    val_acc = accuracy_score(val_labels, val_preds)
    epoch_time = time.time() - start_time

    print("\n" + "=" * 60)
    print(f"Epoch {epoch+1}/{EPOCHS} completed")
    print(f"Train Loss: {train_loss:.4f}")
    print(f"Validation Loss: {val_loss:.4f}  |  Validation Accuracy: {val_acc:.4f}")
    print(f"Time: {epoch_time/60:.2f} minutes")
    print("=" * 60)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        epochs_no_improve = 0
        torch.save(roberta_model.state_dict(), "best_roberta_lr2e5.pt")
        print("Best model saved!")
    else:
        epochs_no_improve += 1
        print(f"No improvement for {epochs_no_improve} epoch(s)")
        if epochs_no_improve >= PATIENCE:
            print(f"\nEarly stopping triggered at epoch {epoch+1}")
            break

print("\nTraining finished. Best validation loss:", round(best_val_loss, 4))
```

Code 16 — RoBERTa Evaluation

```
roberta_model.load_state_dict(torch.load("best_roberta_lr2e5.pt"))
roberta_model.eval()

all_preds, all_labels = [], []
with torch.no_grad():
    for batch in test_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)
        outputs = roberta_model(input_ids=input_ids, attention_mask=attention_mask)
        preds = torch.argmax(outputs.logits, dim=1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

from sklearn.metrics import precision_recall_fscore_support
acc = accuracy_score(all_labels, all_preds)
prec, rec, f1, _ = precision_recall_fscore_support(all_labels, all_preds, average="
    weighted")
print("RoBERTa (lr=2e-5)")
print("Accuracy :", round(acc, 4))
print("Precision:", round(prec, 4))
print("Recall   :", round(rec, 4))
print("F1 Score :", round(f1, 4))
```

Code 17 — Comparison

```
results = pd.DataFrame([
    {
        "Model": "Logistic Regression",
        "Accuracy": lr_acc,
        "Precision": lr_prec,
        "Recall": lr_rec,
        "F1-Score": lr_f1
    },

    {
        "Model": "LSTM",
        "Accuracy": lstm_acc,
        "Precision": lstm_prec,
        "Recall": lstm_rec,
        "F1-Score": lstm_f1
    },

    {
        "Model": "GRU",
        "Accuracy": gru_acc,
        "Precision": gru_prec,
        "Recall": gru_rec,
        "F1-Score": gru_f1
    },

    {
        "Model": "RoBERTa",
        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1-Score": f1
    }
])

results_display = results.copy()

for col in [
    "Accuracy",
```

```
    "Precision",
    "Recall",
    "F1-Score"
]:

    results_display[col] = (
        results_display[col] * 100
    ).round(2)

display(results_display)
```

Code 18 — LR,LSTM & GRU Confution matrix

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

def plot_cm(y_true, y_pred, title, cmap):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt='d', cmap=cmap,
                xticklabels=['Not Bullying','Bullying'],
                yticklabels=['Not Bullying','Bullying'])
    plt.title(title)
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.tight_layout()
    plt.savefig(title.replace(' ','_') + '.png', dpi=150)
    plt.show()

# Logistic Regression
plot_cm(y_test, lr_preds, "Logistic Regression Confusion Matrix", "Blues")

# LSTM
plot_cm(y_test, lstm_preds, "LSTM Confusion Matrix", "Oranges")

# GRU
plot_cm(y_test, gru_preds, "GRU Confusion Matrix", "Greens")
```

Code 19 — Roberta Confusion Matrix + Model Comparison Graph

```
cm = confusion_matrix(
    all_labels,
    all_preds
)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=[
        "Not Bullying",
        "Bullying"
    ],
    yticklabels=[
        "Not Bullying",
        "Bullying"
    ]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("RoBERTa Confusion Matrix")

```

```
plt.show()


results_display.set_index(
    "Model"
)[
    [
        "Accuracy",
        "Precision",
        "Recall",
        "F1-Score"
    ]
].plot(
    kind="bar",
    figsize=(10, 6)
)

plt.title(
    "Model Performance Comparison"
)

plt.ylabel("Score (%)")

plt.ylim(0, 100)

plt.xticks(
    rotation=15
)

plt.show()
```

Code 20 — Binary LIME Explanation — Setup

```
import torch
import numpy as np
from lime.lime_text import LimeTextExplainer
import matplotlib.pyplot as plt

# Define class names
class_names = ["Not-Bullying", "Bullying"]

# Define a prediction function for LIME
def roberta_predict_proba(texts):
    # Ensure roberta_model and roberta_tokenizer are in scope or passed
    global roberta_model, roberta_tokenizer, device

    # Prepare the model for evaluation
    roberta_model.eval()

    # Process texts in batches to be efficient
    # LIME generates many perturbed samples, so batching is important
    batch_size = 32 # You can adjust this batch size
    probabilities = []

    for i in range(0, len(texts), batch_size):
        batch_texts = texts[i:i + batch_size]

        encodings = roberta_tokenizer(
            batch_texts,
            truncation=True,
            padding="max_length",
            max_length=128,
            return_tensors="pt"
        )

        input_ids = encodings["input_ids"].to(device)
```

```
        attention_mask = encodings["attention_mask"].to(device)

        with torch.no_grad():
            outputs = roberta_model(input_ids=input_ids, attention_mask=attention_mask)
            # Apply softmax to logits to get probabilities
            probs = torch.softmax(outputs.logits, dim=1).cpu().numpy()
            probabilities.extend(probs)

    return np.array(probabilities)

# Initialize LIME Explainer
explainer = LimeTextExplainer(class_names=class_names)
```

Code 21 — Binary LIME Explanation — Interactive User Input

```
while True:

    print("\n" + "=" * 65)

    user_text = input(
        "Enter a comment (type 'exit' to stop): "
    )

    # ---------------------------------------
    # EXIT
    # ---------------------------------------

    if user_text.strip().lower() == "exit":

        print("\nSystem stopped.")

        break


    # ---------------------------------------
    # EMPTY INPUT
    # ---------------------------------------

    if not user_text.strip():

        print("\nPlease enter a comment.")

        continue


    # ---------------------------------------
    # BINARY PREDICTION
    # ---------------------------------------

    probabilities = roberta_predict_proba(
        [user_text]
    )[0]

    prediction = np.argmax(
        probabilities
    )

    confidence = (
        probabilities[prediction] * 100
    )


    # ---------------------------------------
    # RESULT
    # ---------------------------------------

```

```
    print("\n" + "-" * 65)

    print("Comment:")
    print(user_text)

    print(
        "\nBinary Prediction:",
        class_names[prediction]
    )

    print(
        "Binary Confidence:",
        f"{confidence:.2f}%"
    )


    # ---------------------------------------
    # LIME EXPLANATION
    # ---------------------------------------

    print(
        "\nGenerating LIME explanation..."
    )

    explanation = explainer.explain_instance(
        user_text,
        roberta_predict_proba,
        num_features=10,
        num_samples=500
    )


    # ---------------------------------------
    # IMPORTANT WORDS
    # ---------------------------------------

    print("\nImportant Words:")

    print("-" * 65)

    for word, weight in explanation.as_list():

        if weight > 0:

            direction = "? Bullying"

        else:

            direction = "? Not-Bullying"


        print(
            f"{word:25s}"
            f"{weight:+.4f} "
            f"{direction}"
        )

    print("-" * 65)


    # ---------------------------------------
    # LIME GRAPH
    # ---------------------------------------

    print(
        "\nLIME Explanation Graph:"
```

```
    )

    explanation.as_pyplot_figure()

    plt.tight_layout()

    plt.show()


    # ------------------------------------
    # NEXT COMMENT
    # ------------------------------------

    print(
        "\n[OK] Explanation completed."
    )

    print(
        "You can enter another comment."
    )
```